

南 开 大 学

本科生毕业论文（设计）

中文题目： 基于图粗化的社交网络分析系统的设计与实现

外文题目： Design and Implementation of Social
Network Analysis System Based on
Graph Coarsening

学号：	2211532
姓名：	石家伊
年级：	2022 级
专业：	信息安全
系别：	信息安全
学院：	密码与网络空间安全学院
指导教师：	温延龙 教授
完成日期：	2026 年 5 月

关于南开大学本科生毕业论文（设计）的声明

本人郑重声明：所呈交的学位论文，是本人在指导教师指导下，进行研究工作所取得的成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或没有公开发表的作品内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律责任由本人承担。

学位论文作者签名：石家伊
2026年 5 月 18 日

本人声明：该学位论文是本人指导学生完成的研究成果，已经审阅过论文的全部内容，并能够保证题目、关键词、摘要部分中英文内容的一致性和准确性。

学位论文指导教师签名：温延斌
2026年 5 月 18 日

摘 要

随着社交网络数据规模的持续增长，直接在原始图上进行图分析和图神经网络训练会面临计算开销大、存储成本高和实验效率低等问题。该文围绕社交网络图数据的高效压缩与下游分类应用，研究并实现了一种基于通用图粗化（Universal Graph Coarsening, UGC）的社交网络图粗化系统。论文首先分析 UGC 方法的基本原理与执行流程，结合单标签和多标签社交网络数据的特点，从节点特征聚合和标签处理两个方面对原始方法进行改进；随后引入实际压缩率、平均特征值误差、狄利克雷能量相对误差、平均标签纯度、低纯度超节点占比和分类准确度等指标，对不同压缩率和不同优化策略下的粗化效果进行实验分析。实验结果表明，改进后的特征聚合方法能够在保持原有粗化结构不变的前提下，增强部分数据集上的超节点特征表达能力，并在高压压缩率或部分社交网络数据集上提升图神经网络（Graph Neural Network, GNN）分类效果。最后，论文设计并实现了面向 UGC 实验流程的图粗化系统，支持数据集管理、参数配置、粗化运行、结果分析、图结构可视化和 GNN 分类测试等功能。研究表明，该系统能够有效支撑社交网络图粗化实验与结果验证，为大规模图数据的压缩表示和图学习任务提供了一种可行方案。

关键词：社交网络；图粗化；通用图粗化 (UGC)；图神经网络 (GNN)；节点分类；数据可视化

Abstract

With the rapid growth of social network data, graph analysis and graph neural network training on original graphs face high computational cost, large storage demand, and low experimental efficiency. This thesis studies and implements a social network graph coarsening system based on Universal Graph Coarsening (UGC). The purpose is to improve the efficiency of graph learning while preserving important structural and semantic information of the original graph.

The thesis first analyzes the basic principle and workflow of UGC. According to the characteristics of single-label and multi-label social network datasets, the original method is improved from two aspects: node feature aggregation and label processing. Several evaluation metrics are used to measure the coarsening quality, including actual compression ratio, average feature value error, relative error of Dirichlet energy, average label purity, proportion of low-purity supernodes, and classification accuracy. Experimental results show that the improved similarity-weighted feature aggregation method can enhance the discriminative representation of supernode features on some datasets, especially under higher compression ratios. It also improves the classification performance of Graph Neural Network (GNN) models on some datasets.

Finally, a graph coarsening system for the UGC experimental process is designed and implemented. The system supports dataset management, parameter configuration, graph coarsening execution, result analysis, graph visualization, and GNN classification testing. The results indicate that the system can effectively support social network graph coarsening experiments and result verification. It provides a feasible solution for compressed representation of large-scale graph data and graph learning tasks.

Key Words: social network; graph coarsening; universal graph coarsening (UGC); graph neural network (GNN); node classification; data visualization

目 录

摘要.....	I
Abstract.....	II
目录.....	III
第一章 绪论.....	1
第一节 研究背景与意义.....	1
第二节 国内外研究现状.....	2
第三节 研究内容与主要工作.....	3
第四节 论文组织结构.....	4
第二章 相关技术.....	5
第一节 图粗化理论.....	5
第二节 UGC 方法原理.....	6
第三节 图神经网络分类方法.....	7
第四节 系统开发技术.....	8
第三章 系统需求分析.....	9
第一节 功能性需求.....	9
第二节 算法处理需求分析.....	10
第三节 非功能性需求.....	14
第四章 系统设计.....	16
第一节 系统总体架构设计.....	16
第二节 核心算法流程设计.....	17
4.2.1 UGC 粗化总体流程设计.....	17
4.2.2 面向单标签数据集的优化设计.....	17
4.2.3 面向多标签数据集的优化设计.....	19
第三节 系统功能模块设计.....	21
4.3.1 数据输入与参数配置模块设计.....	21
4.3.2 图粗化流程运行模块设计.....	22
4.3.3 粗化结果分析模块设计.....	23
4.3.4 图结构可视化模块设计.....	23
4.3.5 GNN 分类测试模块设计.....	24
第四节 数据库设计.....	25
第五章 系统实现.....	27
第一节 系统运行环境配置.....	27

第二节	数据输入与参数配置模块实现.....	27
第三节	图粗化流程运行模块实现.....	30
第四节	粗化结果分析模块实现.....	31
第五节	图结构可视化模块实现.....	34
第六节	GNN 分类测试模块实现.....	35
第六章	系统测试.....	38
第一节	测试环境与测试设置.....	38
第二节	粗化算法有效性测试.....	39
6.2.1	测试设置	39
6.2.2	单标签数据集上的特征聚合优化测试	40
6.2.3	不同 γ 参数对特征加权聚合的影响测试.....	42
6.2.4	多标签数据集上的标签处理策略对比测试	43
6.2.5	粗化算法测试小结	45
第三节	系统功能测试.....	46
第四节	系统性能测试.....	47
6.4.1	页面响应性能	47
6.4.2	核心流程运行性能	47
第七章	总结与展望.....	49
附录		50
参考文献		51
致谢		53

第一章 绪论

第一节 研究背景与意义

随着互联网平台和移动社交媒体的快速发展，社交网络已经成为信息传播、用户交互和群体行为演化的重要载体。微博、Twitter、Facebook 等平台在长期运行过程中积累了大量用户关系、文本属性和行为标签数据，这些数据通常可抽象为由节点、边、属性和标签构成的复杂图结构。围绕此类图数据开展节点分类、社区发现、关系预测和传播分析，能够为社交推荐、舆情监测和用户画像等应用提供支撑，因此社交网络图数据分析已成为图挖掘与图机器学习领域的重要研究方向^[1-2]。

近年来，图神经网络在图结构数据建模任务中取得了较好效果^[3-4]，但随着社交网络规模持续增大，原始图往往具有节点数量多、边连接复杂、属性维度高等特点，直接在原图上进行图学习会带来较高的存储开销和计算成本。同时，社交网络中不同节点可能具有相似的属性模式和连接关系，原始图中存在一定冗余信息。因此，如何在尽可能保留原图关键结构与语义信息的前提下压缩图规模、提升分析效率，成为社交网络分析中的重要问题。

图粗化技术为解决上述问题提供了有效思路。与简单的图采样或删除边删点方法不同，图粗化通常通过将原图中的相似节点聚合为超节点，构造规模更小的粗图，在降低计算复杂度的同时尽量保留原图的结构特征和谱性质，从而为大规模图学习任务提供更高效的数据表示。

然而，真实社交网络数据往往具有标签复杂、语义交叠和多标签共存等特点，节点之间的相似性难以仅由单一结构或标签信息刻画，这对图粗化方法在社交网络场景中的适用性提出了更高要求。因此，结合社交网络数据特点研究图粗化优化方法，并通过系统化方式支撑数据处理、粗化实验、结果分析和分类测试，具有一定研究价值。

从理论意义上看，面向社交网络场景研究图粗化方法，有助于拓展图缩减技术在复杂图数据上的应用范围，并为图学习任务中的高效表示提供参考。从应用意义上看，在较好保留原图关键信息的基础上实现图规模压缩，有助于降低社交网络分析中的计算成本，提高节点分类等任务的执行效率，并为社交推荐、用户

分群、行为分析和舆情监测等应用提供支撑。综上所述，在社交网络规模持续增长、图学习计算代价不断上升的背景下，研究基于图粗化的社交网络分析方法及其系统实现具有较强的理论意义和现实价值。

第二节 国内外研究现状

随着图数据在社交网络、推荐系统、生物信息学和知识图谱等领域中的广泛应用，面向大规模图的高效表示与分析逐渐成为国内外研究热点。现有图缩减研究大致包括图稀疏化、图粗化、图凝缩和图摘要等方向^[5]。其中，图粗化通过将原图节点聚合为超节点构造粗图，能够在降低图规模的同时尽量保持结构特征和谱性质，因此成为连接传统图压缩方法与现代图学习任务的重要方向^[6]。

国外图粗化相关研究起步较早，早期主要集中在图划分、多层次结构组织和谱近似等方面。Kernighan 和 Lin 提出的图划分启发式方法为后续研究奠定了基础^[7]，Karypis 和 Kumar 提出的多层次图划分框架进一步推动了多尺度图组织方法的发展^[8]。随着图机器学习的发展，研究重点逐渐转向面向下游任务的图压缩。Chen、Saad 和 Zhang 对图粗化从科学计算到机器学习的发展路径进行了总结^[6]；Hashemi 等人则从统一视角梳理了图稀疏化、图粗化和图凝缩等方法^[5]。在具体方法上，Loukas 从理论层面研究了图缩减中的谱保持与割保持问题^[9]；Ying 等人提出的 DiffPool 将可微池化思想引入图神经网络任务^[10]；Cai、Wang 和 Wang 提出了数据驱动图粗化方法，体现了图粗化与任务优化结合的趋势^[11]。Kataria、Kumar 和 Jayadeva 提出的通用图粗化（Universal Graph Coarsening, UGC）方法进一步增强了图粗化方法在不同图学习任务中的适用性^[12]。

国内相关研究总体上与国际趋势保持一致，主要关注大规模图计算优化、图神经网络加速和图压缩训练等方向。部分工作虽未直接以“图粗化”为核心概念，但同样围绕高效图学习展开。例如，马煜昕等针对大规模图训练中的特征加载瓶颈，提出基于输入特征稀疏化的图神经网络训练加速方法，在保证模型精度基本稳定的前提下降低了内存占用并提升了训练效率^[13]。总体来看，国内研究在工程实现和应用适配方面具有一定优势，但在复杂标签场景下的粗化质量评价和全流程系统集成方面仍有拓展空间。

在图神经网络与社交网络分析方面，图卷积网络（Graph Convolutional Net-

work, GCN)^[14]、GraphSAGE (SAmple and aggreGatE, GraphSAGE)^[15]、图注意力网络 (Graph Attention Network, GAT)^[16]、图同构网络 (Graph Isomorphism Network, GIN)^[17] 等模型已广泛应用于节点分类、社区识别、关系预测和用户行为分析等任务。然而, 面对大规模社交网络时, 现有图神经网络 (Graph Neural Network, GNN) 模型仍存在训练成本高、推理效率受限等问题。为缓解这些问题, 研究者提出了采样训练、图池化和图缩减等方法, 其中图粗化为降低数据冗余、提高训练效率提供了重要思路^[5-6]。

此外, 真实社交网络中用户通常具有多重兴趣、身份和行为属性, 多标签数据十分常见。相关综述指出, 在标签之间存在交叉、包含和弱边界关系时, 建模标签相关性是提升多标签学习效果的重要途径^[18]。这为多标签图学习提供了基础, 但在图粗化场景下, 如何将多标签信息转化为适合粗化的相似性刻画方式, 仍缺乏统一成熟的方案。

总体来看, 现有研究已为图缩减、图神经网络和多标签图分析提供了较好基础, 但在社交网络场景下, 粗化质量与下游任务性能之间的协同优化、复杂标签信息的有效利用以及全流程分析支持等方面仍有进一步研究空间。基于上述现状, 本文以 UGC 图粗化方法在社交网络场景下的适用性与可用性为切入点展开研究。

第三节 研究内容与主要工作

本文围绕基于 UGC 的社交网络图粗化方法优化与系统实现展开研究。通过分析社交网络数据的结构特征与标签特点, 针对图粗化过程中的关键问题开展实验与改进, 并完成图粗化分析系统的设计与实现。本文的主要工作如下。

1. 图粗化方法效果分析与优化。基于社交网络数据集对 UGC 方法进行实验分析, 发现原始方法在谱性质保持方面表现较好, 但部分数据集上的下游 GNN 分类效果仍有提升空间。针对该问题, 本文从超节点标签和特征聚合方式两个角度进行分析, 并改进特征聚合方法, 以提升粗化图在分类任务中的表现。
2. 粗化质量评价指标补充。针对现有评价方法对超节点内部标签一致性刻画不足的问题, 引入平均标签纯度和低纯度超节点占比两个指标, 从标签一

致性角度辅助评价粗化质量，为实验结果分析提供更细粒度的依据。

3. 多标签数据处理策略比较。面向社交网络中多标签、标签交叠等特点，设计多种标签处理策略，并对不同策略下的异配系数、图粗化结果和下游分类效果进行对比分析，考察多标签处理方式对图粗化效果的影响。
4. 图粗化分析系统设计与实现。结合图粗化实验需求，设计并实现基于 UGC 的社交网络图粗化分析系统，支持数据处理、参数配置、粗化运行、结果导出、GNN 分类测试和可视化展示等功能，为方法验证和后续扩展提供系统支撑。

第四节 论文组织结构

本文共分为七个章节，各章节内容安排如下：

第一章为绪论。本章主要介绍本文的研究背景与意义，梳理国内外相关研究现状，并概述本文的研究内容与主要工作。

第二章为相关技术。本章主要介绍本文研究与系统实现所涉及的关键技术，包括图粗化理论、UGC 方法原理、图神经网络分类方法以及系统开发相关技术。

第三章为系统需求分析。本章从功能性需求、算法处理需求和非功能性需求三个方面对系统进行分析，明确系统在数据输入、参数配置、图粗化运行、结果展示、图结构可视化、GNN 分类测试以及算法扩展等方面应满足的需求。

第四章为系统设计。本章在需求分析的基础上，对系统总体架构、核心算法流程和功能模块进行设计，说明系统各层结构、主要模块划分以及单标签特征聚合优化和多标签处理策略的设计思路。

第五章为系统实现。本章介绍系统运行环境及各主要功能模块的实现过程，包括数据输入与参数配置、图粗化流程运行、粗化结果分析、图结构可视化和 GNN 分类测试等模块。

第六章为系统测试。本章从粗化算法有效性测试、功能性测试和系统性能测试等方面对系统进行验证，分析系统功能完整性、算法处理效果和运行性能。

第七章为总结与展望。本章总结全文的主要研究工作和系统实现成果，分析当前工作中存在的不足，并对后续研究方向进行展望。

第二章 相关技术

第一节 图粗化理论

图粗化是图缩减的重要分支，其基本思想是在尽可能保留原图关键信息的前提下，将多个相似节点聚合为超节点，从而构造规模更小的粗图，以降低存储与计算开销^[5-6]。

从形式化角度看，设原图为 $G = (V, E, X)$ ，其中 V 表示节点集合， E 表示边集合， X 表示节点特征。图粗化的目标是学习一个由原节点到超节点的映射关系，并据此构造粗图 $G_c = (V_c, E_c, X_c)$ ，其中 $|V_c| \ll |V|$ 。理想的粗图不仅应在规模上显著小于原图，还应尽可能保持原图的拓扑结构、谱性质和对下游任务有用的特征信息^[6,9]。图粗化基本流程示意图如图 2.1 所示。

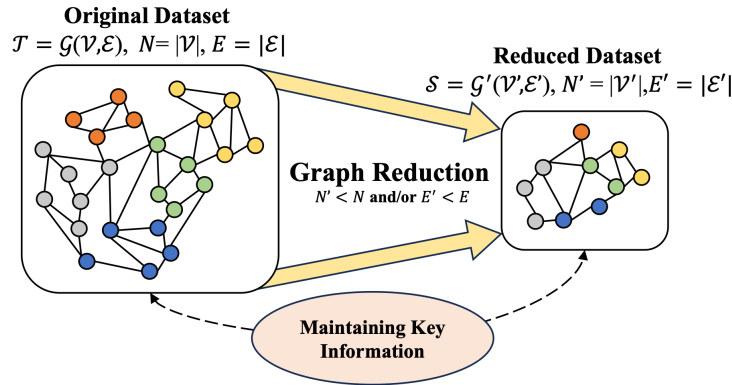


图 2.1 图粗化基本流程示意图^[5]

现有图粗化方法通常可以分为两类：一类更关注谱性质保持，常用于谱聚类、图信号处理和科学计算等场景^[9]；另一类更关注图挖掘与图学习任务中的表示能力，希望粗图能够较好保留社区结构、局部邻域关系和节点特征信息^[6]。在实现机制上，图粗化既可以基于传统的图划分与多层次优化思想实现，也可以通过可学习的分配关系构建层次化图表示^[7-8,10-11]。

对于图粗化结果的评价，现有研究通常从压缩率、结构保持程度和任务性能三个方面展开^[5-6]。其中，压缩率反映图规模降低的程度，结构保持程度反映粗图对原图信息的保留能力，而任务性能则关注粗图在节点分类等下游任务中的表现。UGC 是近年来提出的一种具有代表性的通用图粗化方法，其特点是在统

一框架下综合考虑图结构信息、节点特征信息以及图的异配程度，从而提升方法在不同类型图数据上的适用性^[12]。

第二节 UGC 方法原理

通用图粗化 (Universal Graph Coarsening, UGC) 是一种面向图学习任务的高效图粗化方法，其核心目标是在显著压缩图规模的同时，尽可能保留原图的结构信息、谱性质以及对下游任务有用的语义表示^[12]。与传统图粗化方法主要依赖结构相似性、且多面向同配图设计不同，UGC 在统一框架下同时考虑节点属性与邻接结构信息，并利用图的异配程度调节二者在节点表示中的作用，从而增强了方法对不同类型图数据的适用性^[12]。

从执行流程看，UGC 主要包括增强节点表示构造、基于局部敏感哈希的节点聚合、粗图结构生成以及超节点标签与特征构建等环节，并可进一步结合谱性质保持指标与下游分类性能对粗化效果进行评估^[6,12]。

UGC 的第一步是构造增强节点表示。原论文认为，仅依赖节点特征或仅依赖图结构都难以全面描述节点间相似性，因此将节点属性向量与邻接向量进行融合，并通过异配系数 α 调节二者权重。对任意节点 v_i ，其增强表示可写为

$$F_i = (1 - \alpha) \cdot X_i \oplus \alpha \cdot A_i, \quad (2.1)$$

其中， $\oplus$ 表示向量拼接。

在获得增强表示后，UGC 采用局部敏感哈希 (Locality Sensitive Hashing, LSH) 实现节点分桶与聚合。其基本思想是通过随机投影将表示相近的节点映射到同一哈希桶中，再将同桶节点聚合为同一个超节点。该过程避免了复杂的全局匹配或迭代优化，因此具有较高的计算效率。经过哈希分桶后，可得到原节点到超节点的映射关系 $\pi: V \rightarrow V_c$ ，并进一步形成粗化矩阵 C ^[12]。

在节点聚合完成后，UGC 进一步构造粗图的结构信息。若两个超节点对应的原节点集合之间存在边连接，则在粗图中建立对应的超边，其边权由原图中相关边权累加得到。矩阵形式下，粗图邻接矩阵可表示为

$$A_c = C^T A C \quad (2.2)$$

这意味着粗图结构本质上是对原图连接关系在超节点层面的重新组织与压缩^[12]。

除粗图结构外，UGC 流程还包括超节点标签与超节点特征的生成，这两部分对于后续节点分类任务尤为关键。原论文采用超节点内部节点标签的多数类作为该超节点标签；对于超节点特征，则采用簇内节点特征的平均聚合方式生成粗图特征表示。

第三节 图神经网络分类方法

图神经网络（Graph Neural Networks, GNN）是一类面向图结构数据的深度学习方法，其核心思想是通过节点之间的连接关系对邻域信息进行传播与聚合，从而学习节点表示并完成分类、预测等任务^[14-15]。与传统仅依赖节点属性的分类方法相比，图神经网络能够同时利用节点特征和图结构信息，因此在社交网络分析中具有较强的适用性。

从模型机制上看，现有图神经网络大多遵循“消息传递—邻域聚合—表示更新”的基本框架，即节点先接收邻居节点传递的信息，再通过聚合函数整合邻域表示，最后更新自身嵌入向量^[15-16]。不同模型的差异主要体现在邻域采样方式、聚合函数形式以及权重分配机制上，但其共同目标都是获得更能反映图结构与节点属性的低维表示。

结合当前系统实现，GNN 分类测试模块主要支持 GCN、GraphSAGE、GAT 和 GIN 等模型。其中，GCN 通过归一化邻接矩阵实现局部邻域信息聚合，结构简洁，是节点分类任务中的经典基线模型^[14]；GraphSAGE 采用邻域采样与归纳式聚合机制，更适用于大规模图场景^[15]；GAT 在聚合过程中引入注意力机制，能够为不同邻居分配不同权重^[16]；GIN 则通过更强的聚合函数设计提升对局部结构差异的表达能^[17]。此外，系统还集成了基于传播机制的神经预测的近似个性化传播（Approximate Personalized Propagation of Neural Predictions, APPNP）模型^[19]以及基于三维 Weisfeiler–Lehman 测试思想的 3WL 模型^[20]，用于进一步丰富分类测试与对比分析的模型选择。

在本文的研究中，图神经网络分类方法主要作为粗化效果的下游评价手段。若粗图在显著缩减图规模的同时，仍能保持较好的分类性能，则说明粗化过程较好保留了对任务有用的信息。

第四节 系统开发技术

为实现数据处理、图粗化、结果展示与分类测试等功能,本文系统采用后端服务、图学习计算和前端可视化相结合的开发方案,主要涉及以下技术。

(1) Flask Web 后端开发技术。系统后端采用 Flask 作为 Web 应用框架^[21]。Flask 具有轻量、路由清晰、易于与 Python 算法模块集成等特点,适合构建科研实验原型系统。在本文系统中,Flask 主要负责页面路由、表单请求处理、实验任务触发、运行状态查询和结果文件读取,并通过模板渲染生成数据上传、实验运行、结果展示和 GNN 分类测试等页面。

(2) 图学习计算与分类测试技术。系统的图数据处理、UGC 粗化和 GNN 分类测试主要基于 PyTorch 与 PyTorch Geometric 实现^[22-23]。其中,PyTorch 提供张量计算和模型训练能力,PyTorch Geometric 提供图数据结构、图卷积操作和常用图神经网络模型支持,便于系统对原图与粗图进行统一组织和节点分类测试。

(3) 前端可视化与数据交互技术。系统前端采用 Cytoscape.js 和 Apache ECharts 进行可视化展示。Cytoscape.js 适用于图结构数据的绘制、布局和交互分析^[24],本文利用其实现粗图预览、原图与粗图映射联动、超节点详情查看等功能。Apache ECharts 具有图表类型丰富、配置灵活和交互性强等特点^[25],主要用于展示标定过程曲线、实验统计图表和性能指标变化。

(4) 数据库与文件化数据组织技术。在数据组织与结果管理方面,系统采用数据库与文件系统相结合的混合存储方式。数据库主要用于保存实验记录、运行状态、关键评价指标和 GNN 测试结果摘要,包括实验参数、实验状态、结果目录、分类准确率、训练耗时和测试日志路径等信息,便于系统进行历史结果查询、状态追踪和实验复现。

同时,考虑到图结构数据、节点映射关系、粗化结果文件和运行日志等内容规模较大、格式较复杂,系统仍保留基于 JSON 和 PT 文件的文件化工作流。实验状态文件、粗图结构、超节点映射关系、元数据、可视化数据和日志文件等被保存在对应的输出目录中,并由前端页面通过异步请求读取和展示。

第三章 系统需求分析

第一节 功能性需求

本文设计的基于 UGC 的图粗化分析系统主要面向图粗化实验分析与结果可视化需求，目标是打通从数据输入、参数配置、图粗化运行到结果展示和 GNN 分类测试的完整流程。系统使用者可以通过 Web 页面完成实验配置、流程执行、粗化结果查看以及下游分类测试，从而降低命令行实验操作成本，提高实验过程的可控性和结果分析的直观性。结合系统建设目标，本文将系统功能性需求划分为以下几个方面：

(1) 数据集选择与上传功能。系统应支持用户选择内置数据集，也应支持用户上传自定义数据集。对于内置数据集，系统需要能够读取预先存放的数据目录，并在页面中提供可选数据集列表；对于用户上传的数据集，系统需要支持边集文件、标签文件和特征文件的上传，并对上传文件进行基本格式校验。考虑到本文研究同时涉及单标签和多标签社交网络数据，系统还应能够在数据分析阶段识别数据集的节点规模、边规模、特征维度和标签类型等基本信息，为后续参数配置和流程运行提供依据。

(2) 实验参数配置功能。系统应支持用户在运行实验前配置关键参数，包括标签类型、多标签处理策略、目标压缩率、GNN 模型类型、训练轮数、标定迭代次数等。对于多标签数据集，系统应提供 V1、V2、V3 和 V4 等标签处理策略选项；对于 Jaccard 阈值策略，还应允许用户配置阈值参数。对于优化实验，系统还应提供是否启用相似度加权特征聚合以及相关参数配置入口。

(3) 图粗化流程运行功能。系统应能够根据用户配置自动执行完整的图粗化流程。该流程包括数据分析、数据清洗、Alpha 计算、数据预处理、哈希桶宽标定、UGC 粗化以及结果保存等阶段。在运行过程中，系统需要按照阶段记录执行状态和关键中间结果，并在页面中向用户展示当前运行进度。若某一阶段运行失败，系统应能够给出相应错误信息，便于用户定位问题。

(4) 粗化结果展示功能。系统应能够在实验完成后展示粗化结果概览，包括数据集名称、标签策略、原始节点数、压缩后超节点数、实际压缩率、类别数、平均标签纯度、低纯度超节点占比以及 GNN 测试状态等信息。为进一步展示粗

化图的质量，系统还应展示平均特征值误差、最大特征值误差、最小特征值误差、双曲误差和 Dirichlet Energy 相对误差等指标。

(5) 图结构可视化与交互功能。系统应支持对粗化后图结构进行可视化展示，并提供一定的交互能力。用户应能够查看粗图中的超节点和超边关系，点击超节点查看其包含的原始节点、主标签分布和邻接超节点信息。同时，系统还应支持原图与粗图之间的映射联动，使用户能够观察某个超节点在原图中对应的成员节点及其局部邻域关系。

(6) GNN 分类测试功能。系统应支持在粗化结果基础上进行 GNN 分类测试。用户可以在 GNN 测试页面选择模型类型和训练轮数，并基于当前实验生成的粗图数据重新执行分类测试。系统需要记录训练过程状态，并在测试完成后保存分类准确度、训练时间等结果。

(7) 实验结果保存与导出功能。系统应能够为每次实验创建独立的结果目录，并保存实验状态、实验摘要、运行日志、粗图结构、超节点映射关系、元数据和 GNN 测试结果等文件。同时，系统还应支持将粗化后的数据导出为 PT 格式或文本格式，便于后续复现实验或在其他工具中继续分析。通过文件化保存方式，系统能够较好支持实验记录、结果追踪和后续扩展。

此外，系统还应支持实验记录和 GNN 测试结果的数据库化管理。系统在创建实验时应保存实验基本信息，在实验运行过程中更新实验状态，并在实验完成后记录关键参数、评价指标和结果目录。对于 GNN 分类测试，系统应保存测试模型、训练轮数、分类准确率、训练耗时、测试状态和日志路径等信息，便于结果追踪、历史测试查询和实验复现。

第二节 算法处理需求分析

虽然 UGC 在方法设计上兼顾了图结构信息、节点特征信息与异配特性，并在原始论文中表现出较好的通用性^[12]，但结合本文针对社交网络数据集的实验结果可以发现，原始 UGC 在实际应用中仍存在进一步优化空间。总体来看，这些问题主要体现在以下两个方面：

1. 在单标签社交网络数据集上，UGC 对谱性质的保持效果总体较好，但其下游 GNN 分类性能并不总是理想。

2. 在多标签社交网络数据集上，原始方法并未直接回答多标签信息应如何参与节点关系刻画的问题，不同的标签处理方式会显著影响粗化结果与分类效果。

因此，有必要分别从单标签和多标签两个角度对 UGC 进行进一步分析。本文实验共选取了单标签与多标签两类典型图数据集。其中，单标签数据集包括 Blogcatalog、Flickr 和 Tweibo_sampled，并选取原论文中常用的 Cora 与 Citeseer 作为参照^[12,26]；多标签数据集包括 Facebook 和 Twitter。上述数据集在图规模、结构特性、节点属性维度以及标签空间复杂度等方面存在明显差异，因而能够从不同角度反映 UGC 及其改进方法在社交网络场景中的适用性与泛化能力。各数据集的详细统计信息见附录表 表 1 和 表 2。

(1) 单标签场景下的特征聚合需求。

本文主要采用平均特征值误差和 Dirichlet Energy 相对误差对粗化前后的谱性质保持情况进行刻画^[12]。其中，平均特征值误差用于衡量原图与粗图在拉普拉斯谱上的整体偏差，误差越小，表明粗图对原图谱结构的保持越好；Dirichlet Energy 相对误差则反映粗化后图信号平滑性相对原图的变化程度，其值越小，说明粗图在图信号能量层面越接近原图^[12,27]。

以目标压缩率为 50% 时的单次实验结果为例，Blogcatalog、Flickr 和 Tweibo_sampled 的平均特征值误差与 Dirichlet Energy 相对误差均小于参照数据集 Cora 和 Citeseer，说明粗化后图在谱结构层面与原图仍保持了较高一致性。结合数据集统计信息可知，这些社交网络数据集通常具有较大的节点规模和较高的平均度，在此类复杂图上仍能较好保持谱特性，说明 UGC 在结构压缩方面具有较明显优势。

表 3.1 单标签数据集下 UGC 的粗化结果与分类表现

数据集	实际压缩率 (%)	平均特征值误差	DE 相对误差	分类准确度
Blogcatalog	50.0962	0.283805	0.013225	0.6577
Flickr	50.3102	0.001291	0.004257	0.3162
Tweibo_sampled	49.6600	0.015955	0.003893	0.3885
Cora	48.6337	0.520865	0.121283	0.8306
Citeseer	60.9859	0.614027	0.141152	0.7222

然而，表 3.1 同时表明，在部分数据集上，较好的谱性质保持并未转化为理

想的下游分类表现。例如，Flickr 和 Tweibo_sampled 的平均特征值误差与 DE 相对误差均较低，但分类准确度仍明显低于 Cora 和 Citeseer。由此可见，UGC 在社交网络数据上虽然能够较好保持图的谱结构，但这种结构保持优势并未稳定地转化为节点分类任务上的性能优势。

谱性质保持较好而下游分类任务效果不佳，说明问题可能出现在超节点层面的语义表达上，即粗化后超节点的标签一致性或特征表示方式可能不足以支撑后续分类任务。为进一步刻画超节点内部的标签聚合质量，参考聚类评价中的 purity 思想^[28]，本文引入平均标签纯度 (AvgPurity) 和低纯度超节点占比 (LowPurityRatio) 两个指标。

设粗图共有 M 个超节点，第 i 个超节点记为 S_i ，其包含的原始节点数为 $|S_i|$ 。记超节点 S_i 内出现次数最多的标签所占比例为

$$p_i = \frac{\max_{c \in \mathcal{C}} n_{ic}}{|S_i|}, \quad (3.1)$$

其中， n_{ic} 表示超节点 S_i 中属于类别 c 的原始节点数量， $\mathcal{C}$ 表示标签集合。则平均标签纯度定义为

$$\text{AvgPurity} = \frac{1}{M} \sum_{i=1}^M p_i. \quad (3.2)$$

该指标反映所有超节点在标签一致性上的平均水平，其值越大，说明粗化后超节点内部的标签越集中。

进一步地，给定纯度阈值 τ ，若某超节点满足 $p_i < \tau$ ，则将其视为低纯度超节点。于是，低纯度超节点占比定义为

$$\text{LowPurityRatio}_\tau = \frac{1}{M} \sum_{i=1}^M \mathbb{I}(p_i < \tau), \quad (3.3)$$

其中， $\mathbb{I}(\cdot)$ 为示性函数。该指标用于衡量粗图中标签混杂较严重的超节点所占比例，其值越小，说明粗化结果中存在明显标签混杂问题的超节点越少。本文实验中取 $\tau = 0.7$ 。

结合表 3.2 可以看出，Flickr 和 Tweibo_sampled 的平均标签纯度较高，低纯度超节点占比也相对较低；Blogcatalog 的标签纯度结果与 Cora 基本接近。这说明在单标签数据集上，UGC 的下游分类性能并不完全由超节点内部标签一致性决定，分类效果不足还可能与超节点特征表示方式有关。

表 3.2 单标签数据集下 UGC 的标签纯度

数据集	平均标签纯度	低纯度超节点占比
Blogcatalog	0.759320	0.455457
Flickr	0.782197	0.391605
Tweibo_sampled	0.811290	0.356575
Cora	0.759771	0.450036
Citeseer	0.684832	0.581664

进一步分析可知，原始 UGC 采用简单平均方式生成超节点特征，该方法默认同一超节点内部所有成员节点具有相同贡献。然而，在社交网络数据中，同一超节点内的节点可能存在特征差异、边缘节点或噪声节点。如果直接进行平均聚合，代表性较强的节点特征可能被稀释，从而降低粗图特征对下游分类任务的判别能力。

因此，在单标签社交网络数据处理方面，系统需要支持原始平均聚合与相似度加权聚合两类超节点特征生成方式，并能够计算平均特征值误差、DE 相对误差、平均标签纯度、低纯度超节点占比和分类准确度等指标。通过上述功能，系统可以从谱性质保持、标签一致性和下游分类效果等多个角度评价粗化结果，并为不同特征聚合方式的对比分析提供支撑。

(2) 多标签场景下的标签处理需求。

多标签社交网络中，同一节点通常可以同时具有多个标签，这些标签可能对应用户的兴趣、身份、行为属性或社群关系。

多标签会影响 UGC 中的关键参数与粗化流程。UGC 通过异配系数 α 调节结构信息与特征信息在增强节点表示中的作用，而 α 的计算依赖于边两端节点是否属于同类^[12]。当节点具有多个标签时，节点之间是否属于“同类”不再能够由单一标签直接判断，而不同的同类判定规则会导致不同的 α 取值，并进一步影响增强特征构造、哈希分桶和超节点聚合结果。因此，多标签处理方式并非简单的数据预处理步骤，而是会系统性影响 UGC 整体行为的重要因素。

综上，多标签社交网络给 UGC 带来的主要问题包括：多标签节点之间同类关系难以统一定义，异配系数计算依赖标签处理规则，粗化后超节点标签生成可能造成语义损失，以及不同标签处理方式可能改变下游分类任务的类别空间和难度。基于这些问题，后续章节将进一步设计多种多标签处理策略，并通过实验

比较其对 UGC 粗化结果和分类性能的影响。

因此，在多标签社交网络数据处理方面，系统需要满足以下算法处理需求：首先，应支持多标签数据的识别与预处理，使系统能够区分单标签数据集和多标签数据集，并根据标签类型选择相应处理流程；其次，应支持多种多标签处理策略，包括第一标签策略、标签集合交集策略、位置对齐匹配策略和 Jaccard 阈值策略等，以便从不同角度刻画多标签节点之间的同类关系；再次，应支持在不同标签处理策略下重新计算异配系数 α ，并将其用于后续增强节点表示构造和 UGC 粗化流程；最后，应支持记录并展示不同策略下的类别数、实际压缩率、平均特征值误差、平均标签纯度、低纯度超节点占比和分类准确度等结果，为多标签处理策略的比较和选择提供依据。

第三节 非功能性需求

图粗化实验分析系统的非功能性需求分为以下几部分。

(1) 可用性需求。系统面向图粗化实验分析场景，使用者可能需要频繁调整数据集、标签策略、目标压缩率和模型参数。因此，系统界面应尽可能简洁清晰，使用户能够通过页面完成实验配置和结果查看，减少直接操作命令行脚本的复杂度。系统应在运行过程中展示当前阶段和关键日志信息，使用户能够了解实验是否正常执行；在结果展示阶段，应通过指标卡片、图表和图结构可视化等方式呈现结果，提高实验分析的直观性。

(2) 稳定性需求。图粗化实验通常涉及数据读取、预处理、参数标定、算法运行和模型训练等多个环节，任一环节出错都可能导致实验中断。因此，系统需要具备基本的异常处理能力，在数据格式错误、参数不合法、脚本执行失败或结果文件缺失时，能够给出明确的错误提示。同时，系统应记录各阶段的运行状态和日志信息，便于后续排查问题。对于运行时间较长的实验，系统还应能够持续保存状态，避免用户无法判断任务进度。

(3) 可维护性需求。系统后端应采用清晰的模块划分，避免将数据处理、算法调用、页面路由和结果解析全部耦合在同一逻辑中。本文系统将数据分析、数据清洗、Alpha 计算、预处理、桶宽标定、UGC 粗化和 GNN 测试等环节封装为不同服务模块，有利于后续定位问题和修改功能。前端页面也应按照上传配置、

运行监控、结果展示和 GNN 测试等功能进行组织，使系统结构清晰、代码职责明确。

(4) 可扩展性需求。由于图粗化方法、标签处理策略和 GNN 模型都可能继续扩展，系统应具备一定的扩展能力。具体而言，系统应能够方便地增加新的数据集、标签处理策略、粗化参数、优化方法和分类模型。例如，在多标签处理策略方面，系统应支持在现有 V1 至 V4 策略基础上继续增加新的标签相似性定义；在 GNN 测试方面，系统应支持接入新的图神经网络模型；在可视化方面，系统应能够扩展更多指标图表和交互方式。

(5) 实验可复现性需求。本文系统不仅用于展示粗化结果，也用于支撑实验分析，因此需要保证实验过程和结果具有较好的可复现性。系统应为每次实验生成独立的实验编号，并保存参数配置、运行日志、粗图结构、映射关系、评价指标和 GNN 测试结果等文件。通过结构化保存实验信息，用户可以追溯一次实验的输入参数和输出结果，也可以在后续研究中复用粗图数据和中间结果。

(6) 性能需求。社交网络数据通常具有节点数多、边数多和特征维度高等特点，因此系统需要在算法执行和结果展示方面具有一定性能保障。在后端运行过程中，系统应尽量复用已有脚本和中间结果，减少重复计算；在前端可视化过程中，对于规模较大的粗图或原图映射关系，应采用边筛选、局部展示或异步加载等方式，避免页面渲染压力过大。

第四章 系统设计

第一节 系统总体架构设计

根据前文需求分析,本文设计的图粗化分析系统采用浏览器/服务器(Browser/Server, B/S) 架构。用户通过浏览器访问系统,在前端页面完成数据集选择、实验参数配置、流程运行和结果查看;服务器端基于 Flask 框架接收用户请求,调度 Python 算法流程,并将实验结果以结构化形式返回前端。系统总体架构如 图 4.1 所示。

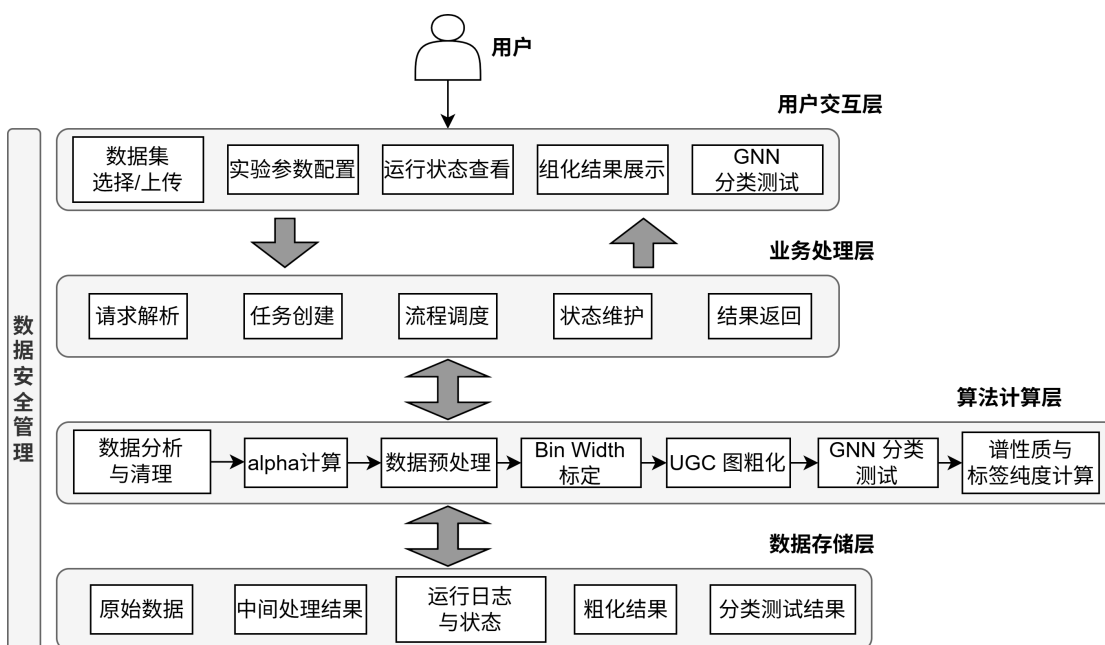


图 4.1 系统总体架构图

系统总体结构可以划分为用户交互层、业务处理层、算法计算层和数据存储层四个部分。

1. 用户交互层主要由 HTML 页面、CSS 样式和 JavaScript 脚本构成,面向系统使用者提供数据上传、参数配置、运行状态查看、粗化结果展示和 GNN 分类测试等交互功能。其中,图结构展示和统计结果展示分别结合 Cytoscape.js 和 ECharts 等前端可视化工具实现,是用户使用系统的直接入口。
2. 业务处理层以 Flask Web 应用为核心,负责连接前端页面与底层算法流程,主要完成请求解析、实验任务创建、流程调度、状态维护和结果返回等工

作。该层将用户在页面中的配置转换为后端可执行的实验任务，并负责向前端反馈运行状态和结果信息。

3. 算法计算层主要由 Python 脚本和图学习相关计算程序组成，是系统实现图粗化分析功能的核心。该层完成数据分析与清洗、Alpha 计算、多标签预处理、Bin Width 标定、UGC 图粗化、谱性质计算、标签纯度统计、特征聚合优化以及 GNN 分类测试等任务，使系统能够自动完成从原始数据到粗化结果的生成。
4. 数据存储层采用数据库与文件系统结合的方式组织数据。数据库用于保存实验记录、运行状态和 GNN 测试结果摘要；文件系统用于保存原始数据、中间处理结果、粗图结构、节点映射、运行日志和可视化文件。二者结合既保证了实验记录的可查询性，也避免了将大规模图数据直接存入数据库带来的额外开销。

第二节 核心算法流程设计

4.2.1 UGC 粗化总体流程设计

根据系统需求，本文将 UGC 粗化流程设计为数据读取与清洗、异配系数计算、增强节点表示构造、Bin Width 标定、局部敏感哈希分桶、粗图结构生成、超节点特征与标签生成以及结果保存等步骤。用户在前端完成数据集和参数配置后，后端按照上述流程依次调用各算法模块，并将中间状态和最终结果写入实验目录。该流程使原本分散的脚本执行过程被组织为统一的系统任务，为后续结果展示、可视化分析和 GNN 分类测试提供输入。

4.2.2 面向单标签数据集的优化设计

根据前文问题分析可知，UGC 在部分单标签社交网络数据集上能够较好保持谱性质，但下游节点分类效果仍不理想。进一步结合平均标签纯度和低纯度超节点占比可以发现，部分数据集的超节点内部标签一致性并未出现明显失控，说明分类性能不足并不能完全归因于标签聚合错误。因此，本文将优化重点从超节点标签层面进一步转向超节点特征表示层面，重点分析原始特征聚合方式是否能够充分保留对分类任务有价值的判别信息。

在原始 UGC 流程中，粗化后超节点的特征由其内部成员节点特征进行平均聚合得到^[12]。设第 i 个超节点为 S_i ，其包含的原始节点集合为

$$S_i = \{v_{i1}, v_{i2}, \dots, v_{im}\}, \quad (4.1)$$

节点 v 的原始特征向量记为 x_v 。则原始平均特征聚合方式可表示为

$$x_i^c = \frac{1}{|S_i|} \sum_{v \in S_i} x_v, \quad (4.2)$$

其中， x_i^c 表示第 i 个超节点的特征表示。该方法实现简单，能够在一定程度上反映超节点内部成员节点的整体特征分布，但其隐含假设是同一超节点内所有成员节点对超节点表示的贡献完全相同。

然而，在社交网络数据中，即使多个节点被聚合到同一超节点，其特征对超节点整体语义的贡献也可能存在差异。一方面，部分节点可能更接近超节点内部的主流特征模式，能够更好代表该超节点；另一方面，部分节点虽然与其他节点被划入同一超节点，但其特征可能处于边缘位置或包含较多噪声。如果对所有成员节点采用完全相同的权重进行平均，可能会削弱代表性节点的贡献，并放大边缘节点或噪声节点对超节点特征的影响，从而影响后续 GNN 分类效果。因此，有必要在超节点特征构建过程中引入成员节点的重要性差异。

基于上述考虑，本文提出一种基于相似度的加权特征聚合方法。该方法首先以超节点内部成员节点特征的均值作为超节点原型表示，记为

$$\bar{x}_i = \frac{1}{|S_i|} \sum_{v \in S_i} x_v. \quad (4.3)$$

随后，计算每个成员节点特征与该超节点原型之间的余弦相似度：

$$s_v = \frac{x_v \cdot \bar{x}_i}{\|x_v\|_2 \|\bar{x}_i\|_2}, \quad v \in S_i. \quad (4.4)$$

其中， s_v 表示节点 v 与其所属超节点原型之间的相似程度。相似度越高，说明该节点越接近超节点内部的主要特征模式，也更适合作为超节点特征表示的主要贡献来源。

在获得相似度后，本文采用 softmax 函数将相似度转换为归一化权重，这种基于相似度的加权方式与注意力机制中的权重归一化思想类似^[16]：

$$w_v = \frac{\exp(\gamma s_v)}{\sum_{u \in S_i} \exp(\gamma s_u)}, \quad v \in S_i, \quad (4.5)$$

其中, γ 为温度参数, 用于调节权重分布的集中程度。当 γ 较小时, 不同成员节点的权重差异相对较小, 加权聚合更接近普通平均; 当 γ 较大时, 相似度较高的节点会获得更大权重, 从而增强代表性节点对超节点特征的影响。

最终, 第 i 个超节点的加权特征表示定义为

$$x_i^c = \sum_{v \in S_i} w_v x_v. \quad (4.6)$$

与原始平均聚合方法相比, 该方法能够根据成员节点与超节点原型的相似程度自适应分配权重, 使超节点特征更加偏向内部主流特征模式, 从而减少特征离群点或噪声节点对粗图表示的影响。

该优化方法并不改变 UGC 已经得到的节点聚合关系和粗图邻接结构, 而仅对超节点特征生成方式进行调整。因此, 该方法能够在保留原有粗化结构和谱性质分析基础的同时, 进一步改善粗图特征表示, 为后续下游 GNN 分类任务提供更具判别性的输入特征。

4.2.3 面向多标签数据集的优化设计

与单标签数据不同, 多标签数据中两个节点是否属于同类并没有唯一标准, 本文针对多标签数据集设计了多种标签处理策略, 用于比较不同同类关系定义对 UGC 粗化结果的影响。

在多标签社交网络中, 一个节点可能同时对应多个标签, 节点标签可以表示为一个标签集合或标签序列。设节点 v_i 的标签集合为

$$L_i = \{l_{i1}, l_{i2}, \dots, l_{ik}\}, \quad (4.7)$$

其中 k 表示节点所具有的标签数量。

UGC 中的异配系数 α 用于衡量图中连接两端节点标签不同的比例, 并进一步影响增强节点表示的构造。对于多标签数据, 本文首先定义一个同类判定函数

$$\delta(v_i, v_j) = \begin{cases} 1, & v_i \text{ 与 } v_j \text{ 被判定为同类,} \\ 0, & v_i \text{ 与 } v_j \text{ 被判定为异类.} \end{cases} \quad (4.8)$$

在此基础上, 多标签场景下的异配系数可写为

$$\alpha = \frac{\sum_{(v_i, v_j) \in E_l} (1 - \delta(v_i, v_j))}{|E_l|}, \quad (4.9)$$

其中, E_l 表示边两端节点均具有有效标签的边集合。由此可见, 不同的同类判定函数会直接影响 α 的取值, 并进一步影响 UGC 的节点表示增强和粗化结果。

本文设计并实现了四种多标签处理策略, 分别记为 V1、V2、V3 和 V4。

(1) V1: 第一标签策略。 V1 策略将每个节点标签序列中的第一个标签作为该节点的代表标签。设节点 v_i 的第一个标签为 l_{i1} , 则两个节点的同类判定规则为

$$\delta_{V1}(v_i, v_j) = \begin{cases} 1, & l_{i1} = l_{j1}, \\ 0, & l_{i1} \neq l_{j1}. \end{cases} \quad (4.10)$$

该策略实现简单, 能够直接将多标签数据转化为单标签形式, 便于复用原始 UGC 流程。但其不足在于仅保留第一个标签会丢弃其余标签信息。当第一个标签不能充分代表节点语义时, 节点间真实的相似关系可能被削弱, 进而影响异配系数计算和后续粗化聚合结果。因此, V1 更适合作为多标签数据单标签化处理的基线策略。

(2) V2: 标签集合交集策略。 V2 策略从标签集合角度判断两个节点是否具有共同标签。若两个节点标签集合存在交集, 则认为二者属于同类; 否则认为二者属于异类。其同类判定规则为

$$\delta_{V2}(v_i, v_j) = \begin{cases} 1, & L_i \cap L_j \neq \emptyset, \\ 0, & L_i \cap L_j = \emptyset. \end{cases} \quad (4.11)$$

该策略能够充分利用多标签重叠信息, 避免 V1 只保留单一标签造成的语义损失。对于标签重叠关系较稀疏的数据, 该策略可以较好地捕捉节点之间潜在的共同语义。但当数据集中标签交叠较普遍时, V2 的判定条件过于宽松, 容易将大量只有弱标签关联的节点都判定为同类, 从而造成类别空间压缩甚至类别塌缩, 降低分类任务的区分度。因此, V2 的实验结果需要结合类别数变化共同分析。

(3) V3: 位置对齐匹配策略。 V3 策略将多标签序列中的标签位置纳入考虑, 只有当两个节点在相同位置上存在相同标签时, 才认为二者具有同类关系。设节点 v_i 和 v_j 的标签序列分别为 $(l_{i1}, l_{i2}, \dots)$ 和 $(l_{j1}, l_{j2}, \dots)$, 则同类判定规则为

$$\delta_{V3}(v_i, v_j) = \begin{cases} 1, & \exists r, l_{ir} = l_{jr}, \\ 0, & \forall r, l_{ir} \neq l_{jr}. \end{cases} \quad (4.12)$$

与 V2 相比，V3 的判定条件更严格，能够避免因任意标签交集导致的同类关系泛化问题。同时，如果标签序列中的位置具有主次含义，V3 能够在一定程度上保留这种标签顺序信息。但是，V3 的有效性依赖于标签顺序本身是否稳定。如果数据集中标签排列并不具有明确语义，或者标签顺序只是由数据存储方式决定，那么位置对齐可能会引入额外约束，使部分本应相似的节点被判定为异类。因此，V3 在类别保持和标签利用之间具有一定折中性。

(4) V4: Jaccard 阈值策略。 V4 策略通过标签集合之间的 Jaccard 相似度判断节点是否属于同类^[29]。两个节点的标签集合相似度定义为

$$J(L_i, L_j) = \frac{|L_i \cap L_j|}{|L_i \cup L_j|}. \quad (4.13)$$

给定阈值 θ ，若 $J(L_i, L_j) \geq \theta$ ，则认为两个节点属于同类；否则认为二者属于异类：

$$\delta_{V4}(v_i, v_j) = \begin{cases} 1, & J(L_i, L_j) \geq \theta, \\ 0, & J(L_i, L_j) < \theta. \end{cases} \quad (4.14)$$

与 V2 相比，V4 不仅关注标签集合是否存在交集，还考虑交集在并集中的相对比例，因此能够排除仅具有少量弱重叠的节点对。与 V1 相比，V4 又能够保留多标签集合中的整体语义信息，而不是只依赖单一代表标签。因此，V4 在标签信息利用和类别空间保持之间具有较好的平衡性。通过调整 Jaccard 阈值，该策略能够适配不同数据集中标签重叠程度的差异。当阈值设置合理时，V4 既可以避免 V1 的语义丢失问题，也可以缓解 V2 的类别空间塌缩问题，因而更适合用于多标签社交网络场景下的 UGC 粗化分析。

第三节 系统功能模块设计

参考系统主要业务流程，本文将系统划分为数据输入与参数配置模块、图粗化流程运行模块、粗化结果分析模块、图结构可视化模块、GNN 分类测试模块和实验结果管理模块。

4.3.1 数据输入与参数配置模块设计

数据输入与参数配置模块是系统运行流程的起点，主要负责实验数据的选择、上传与参数设置。用户进入系统后，可以选择已有数据集，也可以上传自定

义数据集，并根据数据集类型设置标签处理策略、目标压缩率、标定轮数、GNN模型和训练轮数等参数。系统在接收用户输入后，对数据文件和参数进行基本校验，并将配置信息传递给后续图粗化流程模块。

数据输入与参数配置模块的时序图如 图 4.2 所示：

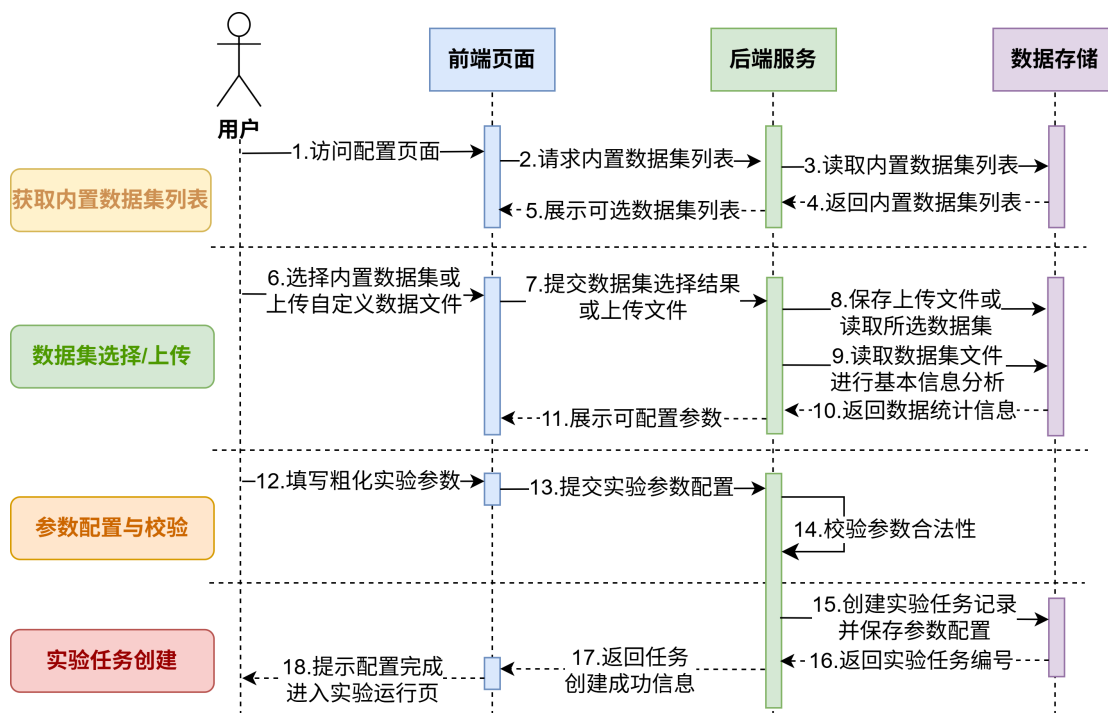


图 4.2 数据输入与参数配置时序图

4.3.2 图粗化流程运行模块设计

图粗化流程运行模块是系统的核心模块，负责根据用户配置自动执行 UGC 图粗化相关流程。该模块将原本分散的数据处理、Alpha 计算、数据预处理、Bin Width 标定和 UGC 粗化执行等步骤进行统一编排。流程运行过程中，系统需要持续记录当前执行阶段、运行日志和关键中间结果，以便前端页面展示任务进度。

图粗化流程运行模块的时序图如 图 4.3 所示：

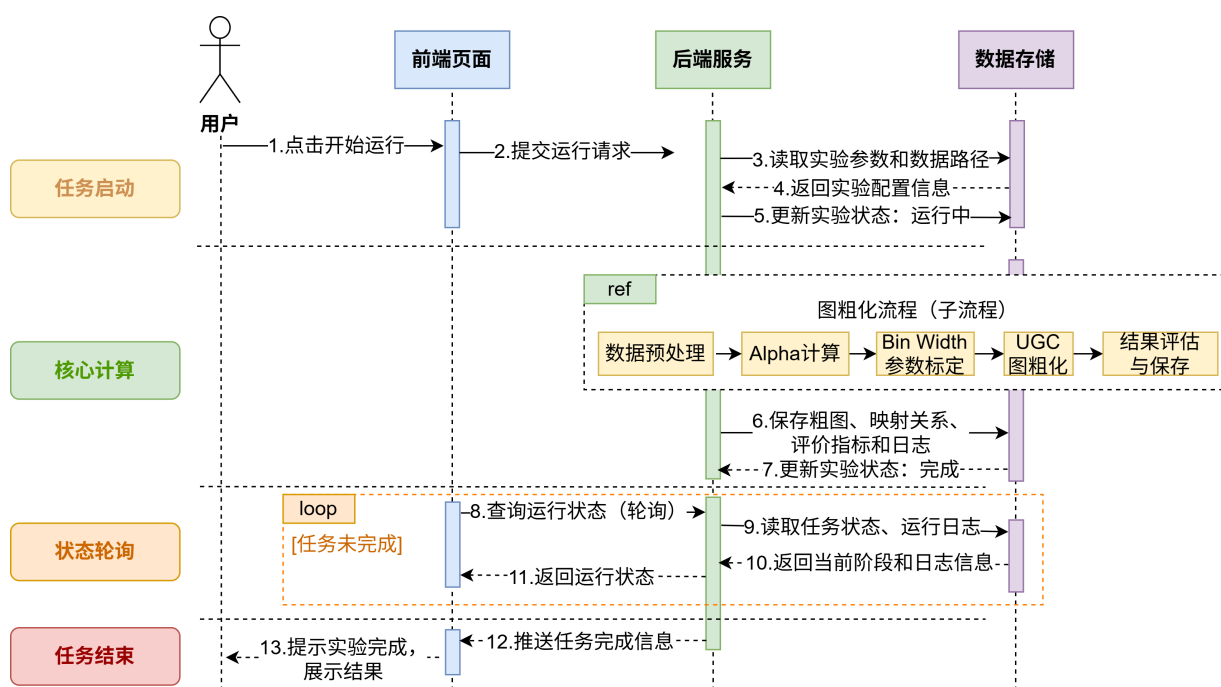


图 4.3 图粗化流程运行时序图

4.3.3 粗化结果分析模块设计

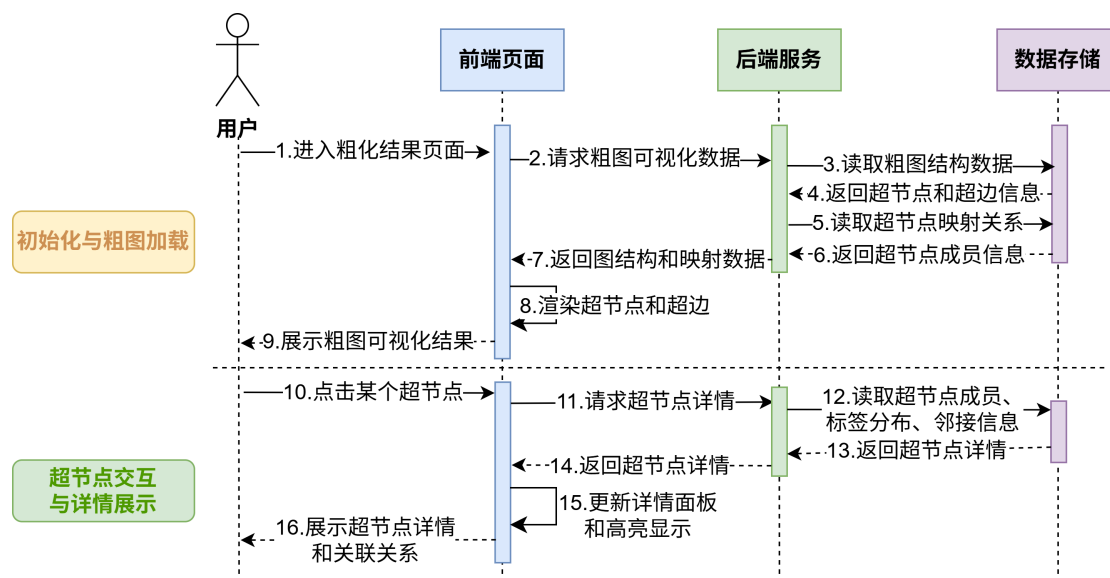
粗化结果分析模块负责对图粗化完成后的输出信息进行整理和评价。该模块不直接执行粗化算法，而是对算法运行结果进行解析、组织和展示，设计重点在于提高结果信息的可读性。

模块主要展示原始节点数、粗化后节点数、实际压缩率、类别数、平均标签纯度、低纯度超节点占比等基础指标；在启用谱性质计算时，还展示平均特征值误差、最大特征值误差、最小特征值误差、双曲误差和 Dirichlet Energy 相对误差等指标。通过这些指标，用户能够从压缩规模、结构保持和标签一致性等角度分析粗化结果。

4.3.4 图结构可视化模块设计

图结构可视化模块用于展示粗化后的图结构及超节点映射关系。用户可以在可视化页面查看粗图中的超节点和超边，并通过点击超节点进一步查看其包含的原始节点、标签分布和邻接信息。该模块使粗化结果不再只体现为表格指标，而能够以图形化方式展示粗图结构和节点聚合关系。

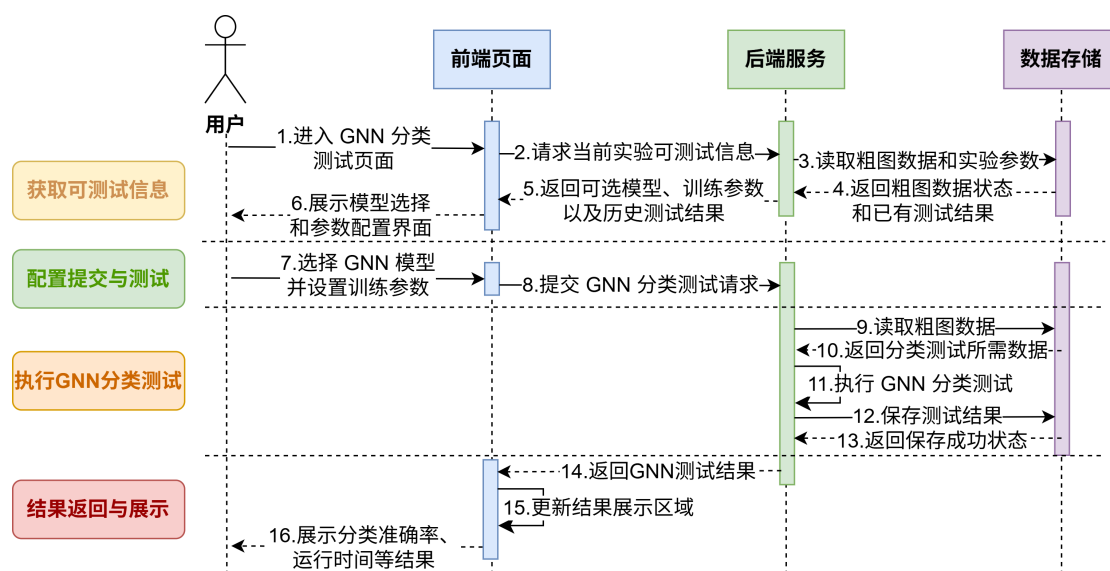
图结构可视化模块的时序图如 图 4.4 所示：



4.3.5 GNN 分类测试模块设计

GNN 分类测试模块用于在粗化图基础上执行下游节点分类实验。用户可以在结果页面或 GNN 测试页面选择分类模型和训练参数，并基于当前实验生成的粗图数据重新执行分类测试。系统在测试完成后，保存分类准确率和训练相关信息，并将结果返回给前端页面展示。

GNN 分类测试模块的时序图如 图 4.5 所示：



第四节 数据库设计

为满足实验记录管理和测试结果追踪需求，系统在原有文件化存储方式基础上引入关系型数据库。数据库主要用于保存实验元信息和 GNN 测试结果摘要，而粗图结构、节点映射、运行日志、可视化数据和模型输入文件仍保存在本地实验目录中。该设计既避免了将大规模图数据直接写入数据库造成的存储和查询压力，又能够通过数据库记录实验状态、关键参数和测试结果，便于系统进行历史结果查询和实验复现。

系统数据库主要包含 `gnn_test_result` 和 `experiment` 两张表，表结构如 表 4.1 和 表 4.2 所示，二者之间构成一对多关系。

表 4.1 `gnn_test_result` 测试结果表设计

字段名	数据类型	说明
<code>id</code>	<code>BIGINT</code>	测试结果唯一编号，主键
<code>experiment_id</code>	<code>BIGINT</code>	关联实验记录编号，外键
<code>model_name</code>	<code>VARCHAR(50)</code>	GNN 模型名称
<code>epochs</code>	<code>INT</code>	训练轮数
<code>accuracy</code>	<code>DOUBLE</code>	分类准确率
<code>train_time</code>	<code>DOUBLE</code>	训练耗时
<code>status</code>	<code>VARCHAR(50)</code>	测试状态
<code>log_path</code>	<code>VARCHAR(255)</code>	测试日志路径
<code>created_at</code>	<code>DATETIME</code>	创建时间

表 4.2 `experiment` 实验记录表设计

字段名	数据类型	说明
<code>id</code>	<code>BIGINT</code>	实验记录唯一编号，主键
<code>dataset_name</code>	<code>VARCHAR(100)</code>	数据集名称
<code>label_type</code>	<code>VARCHAR(50)</code>	标签类型，如 <code>single</code> 或 <code>multi</code>
<code>label_strategy</code>	<code>VARCHAR(50)</code>	标签处理策略，如 <code>V1</code> 、 <code>V2</code> 、 <code>V3</code> 、 <code>V4</code>
<code>target_ratio</code>	<code>DOUBLE</code>	目标压缩率
<code>use_weighted_feature</code>	<code>TINYINT</code>	是否启用相似度加权特征聚合
<code>gamma</code>	<code>DOUBLE</code>	特征加权聚合温度参数
<code>alpha</code>	<code>DOUBLE</code>	异配系数

接下页

续表 4.2

字段名	数据类型	说明
bin_width	DOUBLE	哈希桶宽参数
actual_ratio	DOUBLE	实际压缩率
avg_eigen_error	DOUBLE	平均特征值误差
de_error	DOUBLE	Dirichlet Energy 相对误差
avg_purity	DOUBLE	平均标签纯度
low_purity_ratio	DOUBLE	低纯度超节点占比
status	VARCHAR(50)	实验状态
result_path	VARCHAR(255)	实验结果目录路径
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

第五章 系统实现

第一节 系统运行环境配置

系统运行所涉及的主要软件环境与关键依赖版本如 表 5.1 所示。

表 5.1 系统运行环境与主要依赖版本

名称	版本号
Python	3.11.14
Flask	3.1.3
NumPy	1.24.4
SciPy	1.10.1
PyTorch	2.5.1
PyTorch Geometric	2.4.0
NetworkX	3.1
Matplotlib	3.7.3
Seaborn	0.13.0
Scikit-learn	1.3.1
PyGSP	0.5.1
Cytoscape.js	3.29.2
ECharts	5.x

第二节 数据输入与参数配置模块实现

系统首先通过封面页提供入口，用户进入配置页面后，可以选择系统内置数据集，也可以上传自定义社交网络数据。对于自定义数据集，系统要求用户上传边文件、标签文件和特征文件，并填写数据集名称，如 图 5.1 所示；对于内置数据集，则直接从预设数据集中读取可用选项，如 图 5.2 所示。

该模块在后端由 Flask 路由统一接收请求，首先解析入口模式、数据集名称、标签类型、多标签处理策略等参数；随后根据内置数据集或上传数据集两种不同场景，分别执行数据定位或文件保存操作；最后构造实验表单数据、创建实验记录、写入初始状态文件，并启动后台线程执行后续图粗化流程。其核心实现逻辑如 算法 5.1 所示。

PAGE 1

上传个人数据集并配置 UGC 粗化流程

请输入你的数据集名称，并上传边缘、标签和特征文件。当前后端要求特征文件为 '.npz'。

数据输入

数据集名称

边缘文件

未选择文件

标签文件

未选择文件

特征文件

未选择文件

流程参数

数据集标签类型

多标签

多标签策略

V4 Jaccard Threshold

是否无向化

自动

Target Ratio

50

Jaccard Threshold

0.5

标注轮数

6

相似度加权特征聚合

关闭, 使用均值聚合

开启后, 仅替换 coarse feature 的生成方式: supernode 分组、coarse adjacency 和 coarse label 仍保持原始 UGC 逻辑。

使用提示

- 请先输入数据集名称, 然后同时上传 'edgelist.txt', 'labels.txt' 和 'attrs.npz'。
- 实验记录和结果页会优先展示你输入的数据集名称。
- 当前系统会先执行数据分析、数据清洗、alpha、预处理、bin width 标定, 再完成 UGC 粗化与结果保存。

接下来会生成

- 运行页: 阶段状态、日志、中间结果
- 结果页: 粗化概览、粗图预览、映射关系
- GNN 测试页: 后续单独选择模型和训练轮数再次测试

图 5.1 个人数据集上传页面

PAGE 1

选择内置数据集并配置 UGC 粗化流程

当前模式使用系统内置数据集, 不需要上传文件。你可以直接选择数据集并配置粗化参数。

数据输入

内置数据集

Cora

Cora 使用内置 UGC 数据流程: 无需数据预处理与桶宽标定, 仅支持压缩率 10、30、50、70、90。

流程参数

数据集标签类型

单标签

是否无向化

自动

Target Ratio

50%

相似度加权特征聚合

关闭, 使用均值聚合

开启后, 仅替换 coarse feature 的生成方式: supernode 分组、coarse adjacency 和 coarse label 仍保持原始 UGC 逻辑。

使用提示

- 当前模式会直接使用系统内置数据集, 不需要额外上传文件。
- 'Cora' 与 'Citeseer' 使用内置快捷流程; 'facebook' 走完整 alpha、预处理与桶宽标定流程。
- 当前系统会先执行数据分析、数据清洗、alpha、预处理、bin width 标定, 再完成 UGC 粗化与结果保存。

接下来会生成

- 运行页: 阶段状态、日志、中间结果
- 结果页: 粗化概览、粗图预览、映射关系
- GNN 测试页: 后续单独选择模型和训练轮数再次测试

图 5.2 内置数据集上传页面

算法 5.1 Flask 表单接收与实验创建流程伪代码**Require:** 前端表单请求 *req***Ensure:** 实验编号 *run_id*, 运行状态文件 *state*, 实验运行入口 *url*

```

1: run_id  $\leftarrow$  generate_timestamp_id()
2: (entry_mode, is_json)  $\leftarrow$  parse_request_mode(req)
3: try
4:   (dataset, dataset_display_name, data_root)  $\leftarrow$  prepare_dataset(req, run_id, entry_mode)
5: catch ValueError as e
6:   return request error(e, is_json)
7: end try
8: (label_type, strategy)  $\leftarrow$  parse_label_config(req)
9: if label_type = "single" then
10:   strategy  $\leftarrow$  "v1"
11: end if
12: form_data  $\leftarrow$  build_form_payload(req, dataset, dataset_display_name, label_type, strategy)
13: if invalid_builtin_ratio(dataset, form_data) then
14:   return invalid ratio response
15: end if
16: experiment_id  $\leftarrow$  create_experiment_record(run_id, dataset_display_name, form_data)
17: state  $\leftarrow$  create_initial_state(run_id, form_data, dataset, data_root, experiment_id)
18: save_state(run_id, state)
19: if is_json then
20:   start_experiment_in_background(run_id)
21:   return json response(run_id, running_url, result_url, state_url)
22: else
23:   return redirect(running_url)
24: end if

```

上述流程对应系统中的 `/experiments` 路由实现。对于内置数据集，模块直接读取系统预置的数据集目录，并结合预定义的内容进行参数约束；对于用户上传数据集，模块会先检查边集文件、标签文件和特征文件是否齐全，再将其保存至对应实验子目录中。

在实验创建阶段，系统不会直接阻塞在 Web 请求中执行完整粗化流程，而是先生成唯一实验编号，将实验基础信息和参数配置写入状态文件，再由后台线程异步启动实验。该设计能够避免长时间计算导致页面请求超时，同时便于运行

页持续轮询实验状态、展示阶段日志与中间结果。

第三节 图粗化流程运行模块实现

系统在实验创建时生成独立的实验目录和状态记录写入 `experiment`，并在用户进入运行页面后启动图粗化流程。后端通过统一的流程编排机制，按顺序组织数据分析、数据清洗、Alpha 计算、数据预处理、Bin Width 标定和 UGC 粗化等处理环节，从而将原本分散的脚本执行过程整合为连续的后端运行链路。

图粗化流程运行模块的核心逻辑如 算法 5.2 所示。

算法 5.2 图粗化流程核心伪代码

Require: 数据集 D ，目标压缩率 r ，标签处理策略 S ，标定迭代次数 T

Ensure: 粗图 G_c ，节点映射 M ，评价指标 R

```

1:  $state \leftarrow initialize\_experiment()$ 
2:  $D_{info} \leftarrow analyze\_dataset(D)$ 
3: if  $D$  is uploaded dataset then
4:    $D_{clean} \leftarrow clean\_dataset(D)$ 
5: else
6:    $D_{clean} \leftarrow load\_built\_in\_dataset(D)$ 
7: end if
8:  $Y \leftarrow process\_labels(D_{clean}, S)$ 
9:  $\alpha \leftarrow compute\_alpha(D_{clean}, Y)$ 
10:  $F \leftarrow build\_enhanced\_representation(D_{clean}, \alpha)$ 
11: Initialize candidate bin width  $b$ 
12: for  $t = 1$  to  $T$  do
13:    $G_{temp} \leftarrow run\_ugc\_trial(F, b)$ 
14:    $r_t \leftarrow compute\_compression\_ratio(G_{temp})$ 
15:    $b \leftarrow update\_bin\_width(b, r_t, r)$ 
16: end for
17:  $G_c \leftarrow run\_ugc(D_{clean}, \alpha, b)$ 
18:  $M \leftarrow build\_node\_mapping(G_c)$ 
19:  $A_c \leftarrow build\_coarse\_adjacency(G_c, M)$ 

```

```

20:  $X_c \leftarrow \text{aggregate\_supernode\_features}(D_{\text{clean}}, M)$ 
21:  $Y_c \leftarrow \text{generate\_supernode\_labels}(Y, M)$ 
22:  $R \leftarrow \text{compute\_quality\_metrics}(D_{\text{clean}}, G_c, M)$ 
23:  $\text{update\_experiment\_record}(\text{state}, R)$ 
24: return  $G_c, M, R$ 

```

如果启用相似度加权特征聚合实现优化, 该优化方法核心过程如 算法 5.3 所示。

算法 5.3 超节点相似度加权特征聚合

Require: 节点特征矩阵 X , 超节点成员集合 P , 温度参数 γ

Ensure: 粗图特征矩阵 X_c

```

1:  $X_c \leftarrow \mathbf{0}$ 
2: for  $k \leftarrow 1$  to  $|P|$  do
3:    $V_k \leftarrow P_k$ 
4:   if  $|V_k| = 0$  then
5:     continue
6:   else if  $|V_k| = 1$  then
7:      $X_c[k] \leftarrow X[V_k[0]]$ 
8:     continue
9:   end if
10:   $X_k \leftarrow X[V_k]$ 
11:   $c_k \leftarrow \text{mean}(X_k)$ 
12:   $s \leftarrow \text{cosine\_similarity}(X_k, c_k)$ 
13:   $w \leftarrow \text{softmax}(\gamma \cdot s)$ 
14:   $X_c[k] \leftarrow \sum_i w_i X_k[i]$ 
15: end for
16: return  $X_c$ 

```

其中, 算法 5.3 对应 算法 5.2 中超节点特征聚合步骤的具体实现。

第四节 粗化结果分析模块实现

该模块直接读取实验过程中已经生成的结构化结果文件, 包括实验摘要、谱性质结果和历史测试记录等, 并在页面中以统计卡片和概览面板的形式展示 Alpha、Bin Width、实际压缩率、压缩后超节点数等指标, 如 图 5.3 所示。



图 5.3 基本粗化结果展示页

为进一步展示粗化结果质量,还进一步展示了平均标签纯度、低纯度超节点占比、平均特征值误差、最大特征值误差、双曲误差和 Dirichlet Energy 相对误差等信息,并提供了粗化图下载功能,如图 5.4 所示。



图 5.4 粗化质量指标展示页面

该模块通过 Flask 结果页路由将粗化结果与质量指标返回给前端模板。其核心实现逻辑如算法 5.4 所示。

算法 5.4 结果文件读取与指标返回流程伪代码**Require:** 实验编号 *run_id***Ensure:** 结果页数据载荷对象 *payload*

```

1: summary  $\leftarrow$  load_experiment_summary(run_id)
2: if summary =  $\emptyset$  then
3:   return redirect(running_page)
4: end if
5: state  $\leftarrow$  load_state(run_id)
6: graph_preview  $\leftarrow$  read_json(run_id, "coarse_graph.json")
7: spectral_metrics  $\leftarrow$  read_json(run_id, "spectral_metrics.json")
8: gnn_tests  $\leftarrow$  list_gnn_tests(run_id)
9: dataset_name  $\leftarrow$  resolve_dataset_display_name(state, summary)
10: mapping_node_url  $\leftarrow$  build_mapping_node_api(run_id)
11: mapping_edge_url  $\leftarrow$  build_mapping_edge_api(run_id)
12: payload  $\leftarrow$  {summary, state, dataset_name, graph_preview,
    spectral_metrics, gnn_tests, mapping_node_url, mapping_edge_url}
13: return render_template("result.html", payload)

```

上述流程对应系统中的 `/experiments/<run_id>/result` 路由实现。该接口以实验编号为核心索引，首先读取实验摘要信息，并从中提取 Alpha、Bin Width、压缩率、原始节点数、Supernode 数量、特征维度和类别数等核心指标。在此基础上，系统继续加载粗图预览数据，用于支撑粗图展示与 Supernode 详情联动；读取谱性质评估结果，用于展示平均特征值误差、Dirichlet energy 相对误差等质量指标；同时整理历史 GNN 测试记录，为结果页提供后续分类测试结果的对照信息。

除了结果页整体指标的返回外，该模块还实现了原图与粗图映射关系的按需读取接口。当用户在前端点击某个粗节点或粗边时，系统会根据对应的映射标识加载局部子图数据，并通过接口返回给前端图可视化模块。这种设计使得结果分析不仅能够呈现全局性的粗化效果，还能够支持针对单个 Supernode 或粗边的局部结构进行可解释分析，从而提升系统的交互性与实验结果的可理解性。

第五节 图结构可视化模块实现

系统首先在结果页面加载粗图预览所需的节点、边和统计信息；当用户点击超节点或粗边时，后端再按需返回对应的映射子图数据。这样既保证了页面首次加载的效率，也使后续交互能够展示更细粒度的原图成员关系和局部邻接结构。

在前端实现上，系统利用 Cytoscape.js 渲染粗图中的超节点和超边，并在页面右侧设置详情面板，用于展示被选中超节点的成员节点、标签分布和相邻超节点信息。同时，页面还设计了原图联动视图，用于显示当前超节点对应的原始节点及其上下文邻域，如图 5.5 所示。

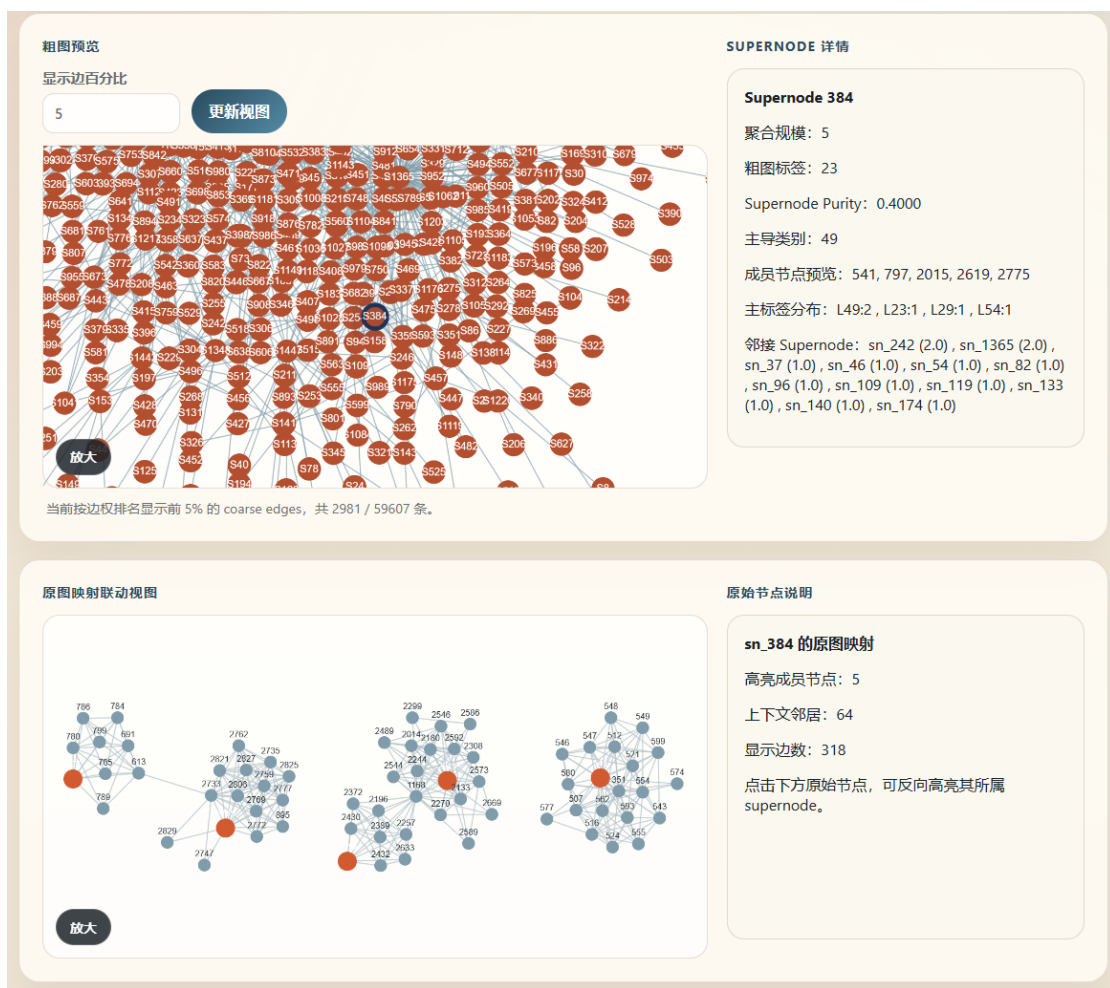


图 5.5 图结构可视化页面

其核心实现逻辑如 算法 5.5 所示。

在该过程中，后端首先遍历粗图中的 Supernode，为每个节点构造可视化对象，写入节点编号、显示标签、聚合规模、类别标签等基础属性；随后进一步补充成员节点数量、主导标签、特征范数、纯度等详情信息，以便前端在点击节点

算法 5.5 Cytoscape 可视化数据构造流程伪代码**Require:** 粗化结果 G_c , 节点映射 M , 质量统计信息 R **Ensure:** 可视化数据对象 V

```

1:  $nodes \leftarrow [], edges \leftarrow [], details \leftarrow \{\}$ 
2: for each supernode  $s$  in  $G_c$  do
3:    $node\_item \leftarrow build\_cytoscape\_node(s)$ 
4:   append  $node\_item$  to  $nodes$ 
5:    $details[s.id] \leftarrow build\_supernode\_detail(s, M, R)$ 
6: end for
7: for each coarse edge  $e$  in  $G_c$  do
8:    $edge\_item \leftarrow build\_cytoscape\_edge(e)$ 
9:   append  $edge\_item$  to  $edges$ 
10: end for
11:  $meta \leftarrow build\_graph\_meta(G_c, R)$ 
12:  $V \leftarrow \{nodes, edges, details, meta\}$ 
13: return  $V$ 

```

时展示对应说明。对于粗边，后端则统一组织边的起点、终点、权重以及唯一标识，并保留后续映射查询所需的关联信息。

此外，为了避免粗图边数过多导致前端渲染负担过重，系统在可视化数据构造阶段还会结合边权重或预设展示比例，对边集合进行筛选，仅保留适合展示的代表性边子集，同时在元信息中记录总边数、实际展示边数和相关说明。这样既能够保持粗图整体结构特征的可感知性，又能够提升页面加载效率与交互流畅性。

第六节 GNN 分类测试模块实现

系统在测试页面中提供模型选择与测试入口，当前支持的模型包括 GCN、GraphSAGE、GAT、GIN、APNP 和 3WL 等。用户在页面中选择模型类型和训练轮数后，前端将测试请求提交至后端，后端读取当前实验对应的粗图数据和实验参数，并启动相应的分类测试流程，测试记录写入 `gnn_test_result`。

为了改善交互体验，系统在该页面中设计了异步提交与轮询更新机制。用户点击“开始测试”后，前端页面会显示加载遮罩，并持续从后端获取当前测试状态和训练轮次，如图 5.6 所示。



图 5.6 GNN 测试进行中页面

以 GCN 为例，其训练过程如 算法 5.6 所示。

算法 5.6 GCN 分类训练核心流程伪代码

Require: 粗图数据 $G_c = (X_c, A_c, Y_c)$ ，训练轮数 E ，模型参数 Θ

Ensure: 分类准确率 Acc ，训练耗时 T

- 1: initialize GCN model f_Θ
- 2: initialize optimizer opt
- 3: $t_{start} \leftarrow current_time()$
- 4: **for** $epoch = 1$ to E **do**
- 5: set model to training mode
- 6: $\hat{Y} \leftarrow f_\Theta(X_c, A_c)$
- 7: $loss \leftarrow cross_entropy(\hat{Y}_{train}, Y_{train})$
- 8: clear optimizer gradients
- 9: backpropagate $loss$
- 10: update Θ with opt
- 11: **end for**
- 12: set model to evaluation mode
- 13: $\hat{Y} \leftarrow f_\Theta(X_c, A_c)$
- 14: $Acc \leftarrow evaluate(\hat{Y}_{test}, Y_{test})$

```
15:  $T \leftarrow \text{current\_time}() - t_{start}$   
16: return  $Acc, T$ 
```

在该过程中，系统首先读取粗化后的节点特征、粗邻接关系和节点标签，并据此构造 GCN 所需的输入图数据。随后初始化图卷积网络及优化器，在训练阶段反复执行前向传播、损失计算、梯度反向传播和参数更新等步骤。

训练完成后，系统会切换模型到评估模式，在测试集上计算分类准确率，并统计本次训练与推理所消耗的总时间。相关结果将被保存为结构化测试记录，供结果页统一展示与横向对比分析。

测试完成后，页面自动刷新以展示最新结果。系统在页面中同时保留最近一次测试结果和历史记录列表，便于用户比较不同模型或不同训练轮数下的分类表现，如 图 5.7 所示。

PAGE 4

返回结果页

GNN 测试页面

数据集: facebook。基于刚刚确定的粗化参数，切换 GNN 模型和训练轮数再次测试。

测试参数

GNN 模型

GCN

训练轮数

10

开始测试

当前粗化参数

数据集	策略
facebook	v4
Alpha	Bin Width
0.3041	0.011240
实际压缩率	
49.83%	

最近一次测试结果

模型	训练轮数
gat	10
压缩率	平均准确率
49.83%	0.7979

历史记录

- gnn_20260424_164422 | gat | 10 epochs | acc 0.7979
- gnn_20260424_164017 | gcn | 10 epochs | acc 0.6287

图 5.7 GNN 分类测试页面

第六章 系统测试

第一节 测试环境与测试设置

本章围绕 UGC 图粗化系统的前后端联动功能、数据库记录能力以及运行性能开展测试。测试工作主要在本地 Windows 开发环境下完成，系统运行于 Conda 虚拟环境 `ugc_cuda` 中，后端数据库采用 MySQL 8.0.19。核心软件环境如下所示：

表 6.1 系统测试的软件环境

类别	名称	版本/说明
运行环境	Python	3.11.14
Web 框架	Flask	3.1.3
深度学习框架	PyTorch	2.5.1
CUDA 支持	CUDA	12.4
数据库	MySQL	8.0.19
数据库驱动	PyMySQL	1.1.2
GPU 设备	NVIDIA GPU	GeForce RTX 2050

测试设置方面，为保证测试结果可复现，本章采用如下统一设置：

1. 内置数据集测试使用系统自带的 Cora 与 Citeseer 数据集。
2. 非内置数据集测试使用 `blogcatalog` 与 `flickr` 数据集，并按系统上传格式组织为 `edgelist.txt`、`labels.txt` 与 `attrs.npz`。
3. 页面响应测试采用 Flask 测试客户端重复请求 10 次统计平均响应时间与最大响应时间。
4. 核心流程性能测试中，内置单标签数据集采用系统内置快捷流程；多标签外部数据集采用 V4 策略、Jaccard threshold=0.5，并将 Bin Width 标定迭代次数设为 2 轮，以控制整组测试时长。
5. GNN 性能测试统一采用 GCN 模型；功能测试中的 GNN 用例按照系统需求执行 500 轮训练，性能测试表中的 GNN 耗时采用 50 轮训练结果统计。

第二节 粗化算法有效性测试

6.2.1 测试设置

本文实验使用的各数据集详细统计信息见附录表 表 1 和 表 2。

本文将节点分类任务作为评价粗化图质量的下游任务。具体流程为：首先对原始图进行 UGC 粗化，得到规模更小的粗图，然后在粗化图上训练 GNN 分类模型，最后将训练得到的模型用于原图节点分类任务，并以原图测试节点上的分类准确率作为评价指标。

在 GNN 分类测试方面，本文主要采用 GCN 作为下游分类模型，训练轮数统一设置为 500 轮。

这种设置的意义在于，粗化图并不是最终目的，而是希望用更小规模的图近似原图。如果粗化过程中能够较好保留原图的结构关系、节点特征和标签语义，那么模型在粗图上学习到的分类规律应当能够迁移到原图节点分类任务中；反之，如果粗化图丢失了过多有效信息，则原图上的分类准确率会下降。因此，分类准确率可以作为衡量粗化图是否有效保留原图任务相关信息的重要指标。

实验中主要采用以下评价指标：

1. **实际压缩率**：表示粗化后节点规模相对于原图节点规模的减少比例，用于衡量图规模压缩程度。
2. **平均特征值误差**：衡量原图与粗图在拉普拉斯谱上的平均偏差，用于评价粗图对原图谱性质的保持程度。
3. **DE 相对误差**：即 Dirichlet Energy 相对误差，用于衡量粗化前后图信号能量的变化程度。
4. **平均标签纯度**：表示粗图中超节点内部标签一致性的平均水平，数值越高说明超节点内部标签越集中。
5. **低纯度超节点占比**：表示标签纯度低于阈值的超节点比例，本文取纯度阈值为 0.7，该指标越低说明标签混杂问题越少。
6. **分类准确度**：表示在粗图上进行 GCN 节点分类测试得到的 Accuracy，用于衡量粗图对下游分类任务的支持能力。

本文主要设置目标压缩率为 50% 和 70% 两组实验。其中，50% 压缩率用于

观察中等压缩强度下不同方法的差异，70% 压缩率用于观察较高压缩强度下粗化信息损失和优化方法作用的变化。

6.2.2 单标签数据集上的特征聚合优化测试

本组实验用于比较原始 UGC 平均特征聚合方式与本文提出的相似度加权特征聚合方式。由于本文的优化方法只改变超节点特征生成方式，不改变 UGC 已得到的节点分组、粗图邻接结构和超节点标签，因此两种方法在实际压缩率、平均特征值误差、平均标签纯度和低纯度超节点占比等结构相关指标上保持一致，差异主要体现在 DE 相对误差和 GCN 分类准确度上。

(1) 目标压缩率为 50% 时的实验结果。

表 6.2 给出了目标压缩率为 50% 时，原始 UGC 与相似度加权特征聚合方法在单标签数据集上的对比结果。其中，相似度加权方法的温度参数设置为 $\gamma = 5$ 。

表 6.2 单标签数据集上原始 UGC 与特征加权方法对比结果（目标压缩率 50%）

数据集	方法	实际压缩率 (%)	平均特征值误差	DE 相对误差	平均标签纯度	低纯度超节点占比	Accuracy
Cora	原始 UGC	48.6337	0.520865	0.121283	0.759771	0.450036	0.8306
	特征加权 $\gamma = 5$	一致	一致	0.424387	一致	一致	0.8029
Citeseer	原始 UGC	60.9859	0.614027	0.141152	0.684832	0.581664	0.7222
	特征加权 $\gamma = 5$	一致	一致	0.513929	一致	一致	0.7207
Blogcatalog	原始 UGC	50.0962	0.283805	0.013225	0.759320	0.455457	0.6577
	特征加权 $\gamma = 5$	一致	一致	0.251235	一致	一致	0.6606
Flickr	原始 UGC	50.3102	0.001291	0.004257	0.782197	0.391605	0.3162
	特征加权 $\gamma = 5$	一致	一致	0.023342	一致	一致	0.3056
Tweibo_sampled	原始 UGC	49.6600	0.015955	0.003893	0.811290	0.356575	0.3885
	特征加权 $\gamma = 5$	一致	一致	0.033669	一致	一致	0.4045

(2) 目标压缩率为 70% 时的实验结果。

表 6.3 给出了目标压缩率为 70% 时，原始 UGC 与相似度加权特征聚合方法在单标签数据集上的对比结果。其中，相似度加权方法的温度参数设置为 $\gamma = 5$ 。

综合 表 6.2 和 表 6.3 可以看出，所有数据集在采用特征加权后，DE 相对误差均出现上升，说明加权聚合改变了粗图特征信号的能量分布，降低了粗化前后图信号能量的一致性。原始平均聚合更接近超节点内部特征均值，因此具有更低的 DE 相对误差；相似度加权聚合则更强调与超节点中心相似的节点特征，能够增强代表性节点的贡献，但会牺牲部分特征信号能量保持性。

从分类准确度看，特征加权的效果与压缩率和数据集特征有关。在目标压缩

表 6.3 单标签数据集上原始 UGC 与特征加权方法对比结果（目标压缩率 70%）

数据集	方法	实际压缩率 (%)	平均特征值误差	DE 相对误差	平均标签纯度	低纯度超节点占比	Accuracy
Cora	原始 UGC	74.4461	1.432789	0.213337	0.617956	0.669075	0.7698
	特征加权 $\gamma = 5$	一致	一致	0.666863	一致	一致	0.7643
Citeseer	原始 UGC	78.5993	1.261886	0.193681	0.569294	0.738764	0.6321
	特征加权 $\gamma = 5$	一致	一致	0.700119	一致	一致	0.6682
Blogcatalog	原始 UGC	70.0539	0.759653	0.021763	0.654821	0.617609	0.3548
	特征加权 $\gamma = 5$	一致	一致	0.410714	一致	一致	0.3731
Flickr	原始 UGC	70.0726	0.026375	0.008936	0.713068	0.487428	0.2323
	特征加权 $\gamma = 5$	一致	一致	0.052157	一致	一致	0.2337
Tweibo_sampled	原始 UGC	69.7300	0.069812	0.012779	0.753843	0.446977	0.3785
	特征加权 $\gamma = 5$	一致	一致	0.085133	一致	一致	0.3950

率为 50% 时，特征加权仅在 Blogcatalog 和 Tweibo_sampled 上取得提升，而 Cora、Citeseer 和 Flickr 未获得改善；当目标压缩率提高到 70% 后，除 Cora 外，其余数据集的 Accuracy 均有所提升。其中，Citeseer 从 0.6321 提升到 0.6682，Blogcatalog 从 0.3548 提升到 0.3731，Flickr 从 0.2323 小幅提升到 0.2337，Tweibo_sampled 从 0.3785 提升到 0.3950。该结果说明，在较高压缩率下，每个超节点聚合的原始节点更多，平均聚合造成的特征稀释问题更加明显，因此相似度加权更容易发挥作用。

不同数据集的表现也存在差异。Blogcatalog 和 Tweibo_sampled 在两种压缩率下均获得提升，说明这类社交网络数据中可能存在较多边缘节点或噪声特征，加权聚合能够突出超节点内部更具代表性的特征信息。Cora 在两种压缩率下均出现下降，说明对于原始特征表达较稳定的数据集，平均聚合可能已经能够较好地保留分类信息，进一步加权反而会引入特征偏移。Flickr 在 50% 压缩率下下降、70% 压缩率下仅小幅提升，说明其分类性能瓶颈可能并不主要来自特征平均聚合。

综上，原始平均聚合与相似度加权聚合之间存在一定权衡：前者更有利于保持图信号能量一致性，后者更有利于在高压压缩率或超节点内部特征差异较大的情况下增强分类判别性。因此，本文提出的相似度加权特征聚合方法针对分类任务，在高压压缩率和部分社交网络数据集上具有更明显的优化作用。

6.2.3 不同 γ 参数对特征加权聚合的影响测试

相似度加权特征聚合中的参数 γ 用于控制权重分布的集中程度。为分析该参数对分类结果的影响，本文在目标压缩率为 50% 的条件下，分别设置 $\gamma = 5$ 、 $\gamma = 10$ 和 $\gamma = 15$ 进行实验，并与原始 UGC 进行对比。实验结果如表 6.4 所示。

表 6.4 不同 γ 参数下单标签数据集的分类准确度对比（目标压缩率 50%）

数据集	原始 UGC	$\gamma = 5$	$\gamma = 10$	$\gamma = 15$
Cora	0.8306	0.8029	0.7919	0.7956
Citeseer	0.7222	0.7207	0.7147	0.7057
Blogcatalog	0.6577	0.6606	0.6654	0.6721
Flickr	0.3162	0.3056	0.3043	0.3050
Tweibo_sampled	0.3885	0.4045	0.3980	0.3910

由表 6.4 可知，不同数据集对 γ 的敏感性存在差异。对于 Blogcatalog，随着 γ 从 5 增大到 15，分类准确度从 0.6606 逐步提升到 0.6721，均高于原始 UGC 的 0.6577。这说明在 Blogcatalog 数据集上，更强调与超节点原型相似的中心节点，有助于形成更具代表性的超节点特征表示。

对于 Tweibo_sampled， $\gamma = 5$ 时分类准确度最高，为 0.4045；当 γ 继续增大到 10 和 15 时，准确度分别下降到 0.3980 和 0.3910，但仍接近或高于原始 UGC。这说明 Tweibo_sampled 更适合较温和的加权方式。若 γ 过大，权重会过度集中于少数中心节点，可能削弱超节点内部其他有用信息的贡献，从而影响分类效果。

对于 Cora、Citeseer 和 Flickr，特征加权并未带来明显提升，甚至出现一定下降。这表明 γ 参数并不存在对所有数据集都最优的统一取值。较小的 γ 使加权聚合更接近平均聚合，较大的 γ 则更强调中心节点。若超节点内部本身较为均质，或者平均聚合已经能够较好表达超节点语义，则进一步加权可能不会带来收益；若超节点内部存在一定噪声或边缘节点，适当增大 γ 则可能提升超节点特征的判别性。

6.2.4 多标签数据集上的标签处理策略对比测试

根据前文设计, 本文分别采用 V1、V2、V3 和 V4 四种策略处理多标签信息, 并比较不同策略对类别数、异配系数 α 、粗化质量和分类准确度的影响。为观察不同压缩强度下策略表现的稳定性, 本文分别在目标压缩率为 50% 和 70% 的条件下进行实验。

(1) 目标压缩率为 50% 时的实验结果。

表 6.5 给出了 Facebook 和 Twitter 数据集在目标压缩率为 50% 时, 不同多标签处理策略下的实验结果。

表 6.5 多标签数据集上不同标签处理策略对比结果 (目标压缩率 50%)

数据集	策略	类别数	α	实际压缩率 (%)	平均特征值误差	平均标签纯度	低纯度超节点占比	Accuracy
Facebook	V1	151	0.355372	49.5845	0.262050	0.720965	0.464973	0.4680
Facebook	V2	1	0.117984	49.9654	0.377434	1.000000	0.000000	1.0000
Facebook	V3	73	0.330835	50.1731	0.246427	0.804008	0.362752	0.7237
Facebook	V4	72	0.304136	49.8269	0.267264	0.768043	0.425811	0.8342
Twitter	V1	117	0.517925	49.6208	0.167936	0.730012	0.454839	0.4189
Twitter	V2	16	0.311798	50.1083	0.260970	0.871917	0.247557	0.9108
Twitter	V3	53	0.503067	49.8375	0.228128	0.781948	0.390929	0.5622
Twitter	V4	33	0.431451	49.4583	0.208424	0.783785	0.399786	0.8216

(2) 目标压缩率为 70% 时的实验结果。

表 6.6 给出了 Facebook 和 Twitter 数据集在目标压缩率为 70% 时, 不同多标签处理策略下的实验结果。

表 6.6 多标签数据集上不同标签处理策略对比结果 (目标压缩率 70%)

数据集	策略	类别数	α	实际压缩率 (%)	平均特征值误差	平均标签纯度	低纯度超节点占比	Accuracy
Facebook	V1	151	0.355372	69.5983	0.620610	0.594789	0.619590	0.3851
Facebook	V2	1	0.117984	69.4598	0.813370	1.000000	0.000000	1.0000
Facebook	V3	73	0.330835	70.1177	0.683031	0.764386	0.426419	0.6822
Facebook	V4	72	0.304136	70.4640	0.732436	0.669942	0.563892	0.7427
Twitter	V1	117	0.517925	70.0975	0.496431	0.583840	0.632246	0.3351
Twitter	V2	16	0.311798	69.9350	0.542895	0.797799	0.356757	0.9081
Twitter	V3	53	0.503067	70.2059	0.433068	0.665902	0.572727	0.5081
Twitter	V4	33	0.431451	70.3142	0.508390	0.658593	0.594891	0.7595

综合 表 6.5 和 表 6.6 可以看出, 50% 和 70% 两种压缩率下, 各多标签处理策略的相对表现基本一致。

V1 策略保留的类别数最多，但分类准确度最低，且平均标签纯度较低、低纯度超节点占比较高。这说明仅使用第一个标签虽然能够避免类别空间被压缩，但会丢弃节点的其他标签信息，使节点间潜在的语义关联无法被充分利用。对于 Facebook 和 Twitter 这类多标签社交网络数据，用户属性或兴趣往往由多个标签共同刻画，单一标签难以完整表示节点语义，因此 V1 在分类任务中的表现相对较弱。

V2 策略在两种压缩率下均取得最高 Accuracy，尤其在 Facebook 数据集上 Accuracy 达到 1.0000。但结合类别数可以发现，Facebook 在 V2 策略下类别数被压缩为 1，分类任务实际上退化为单类别判断。因此，V2 的高准确率并不能直接说明该策略具有最优的粗化效果，而是与类别空间塌缩密切相关。其原因在于 V2 只要两个节点存在任意共同标签就判定为同类，当标签集合交叠较多时，大量节点会被合并到相同类别空间中，从而降低了分类任务难度，但也削弱了对多标签语义差异的刻画能力。

V3 策略在类别数保持和标签语义利用之间表现出一定折中性。与 V1 相比，V3 能够利用更多标签信息；与 V2 相比，V3 通过位置对齐限制了过度宽松的同类判定，因此能够避免严重的类别空间塌缩。从实验结果看，V3 在 Facebook 和 Twitter 上的分类准确度均高于 V1，说明引入更多标签信息有助于改善多标签节点之间的关系刻画。但是，V3 的效果仍受标签顺序可靠性的影响。如果标签序列位置不能稳定反映标签主次关系，则位置匹配可能无法充分捕捉节点间真实的语义相似性。

V4 策略整体表现较为均衡。该策略通过 Jaccard 相似度衡量标签集合的重叠比例，既避免了 V1 只保留单一标签造成的信息损失，也缓解了 V2 因任意交集判定导致的类别空间塌缩问题。并且从结果看，V4 的分类准确度在两个数据集和两种压缩率下均高于 V3。例如，Facebook 上 V4 在 50% 和 70% 压缩率下的 Accuracy 分别为 0.8342 和 0.7427，高于 V3 的 0.7237 和 0.6822；Twitter 上 V4 分别达到 0.8216 和 0.7595，也明显高于 V3。说明基于标签集合相似度的 V4 策略比基于标签位置匹配的 V3 策略更稳定。

此外，不同策略下 α 差异明显，说明多标签处理方式不仅影响类别划分，也会改变 UGC 中结构信息与特征信息的融合比例。

综上，多标签策略的优劣不能仅依据 Accuracy 判断，而应结合类别数、 α 、

标签纯度和低纯度超节点占比综合分析。V2 虽然准确率最高，但存在类别空间塌缩风险；V1 对多标签语义利用不足；V3 有一定改进，但稳定性不如 V4。因此，V4 在保留多标签语义和避免过度合并之间取得了较好的平衡，更适合作为本文多标签数据集上的主要处理策略。

6.2.5 粗化算法测试小结

本节围绕单标签和多标签数据集，对 UGC 粗化效果、相似度加权特征聚合方法以及多标签处理策略进行了实验分析。结果表明，UGC 能够有效压缩图规模，但粗化质量不仅受谱性质保持影响，还与超节点特征表达和标签处理方式密切相关。

在单标签实验中，相似度加权特征聚合不改变节点分组、粗图结构和标签组成，因此结构相关指标保持一致。该方法会提高 DE 相对误差，说明其牺牲了一定的图信号能量一致性；但在 Blogcatalog 和 Tweibo_sampled 等社交网络数据集上能够提升分类准确度，且在 70% 压缩率下作用更明显。这表明相似度加权聚合在高压缩率或超节点内部特征差异较大的场景下具有一定优势。

在参数实验中， γ 控制相似度权重的集中程度。不同数据集对 γ 的敏感性不同，说明不存在统一最优取值。实验结果表明，较大的 γ 更适合 Blogcatalog，而 Tweibo_sampled 在 $\gamma = 5$ 时表现较好，因此该参数需要结合数据集特征和粗化结果进行选择。

在多标签实验中，不同标签处理策略会显著影响类别数、异配系数 α 、标签纯度和分类准确度。V1 对多标签语义利用不足，V2 存在类别空间塌缩问题，V3 有一定改进但稳定性有限。相比之下，V4 基于标签集合相似度，在两种压缩率下均表现出较好的综合效果，更适合作为本文多标签数据集的处理策略。

综上，实验结果验证了前文问题分析的合理性。相似度加权特征聚合体现了图信号能量保持与分类判别性之间的权衡，多标签处理策略则需要避免类别空间塌缩，并结合多项指标进行综合评价。

第三节 系统功能测试

根据系统主要业务流程，本文设计了如表 表 6.7 所示的功能测试用例，覆盖数据输入、参数配置、实验创建、图粗化运行、结果展示、图可视化、GNN 测试和异常输入处理等功能。

表 6.7 系统功能测试用例与结果

序号	测试功能	测试内容	实际结果摘要	是否通过
1	内置数据集选择	选择 Cora 并进入配置页	页面成功加载，且在配置页中可见 Cora 选项	是
2	自定义数据集上传	上传边、特征、标签文件	blogcatalog 三个输入文件保存成功，并通过基础完整性校验	是
3	参数配置	设置目标压缩率、标签策略、GNN 模型	参数被正确写入实验状态	是
4	实验记录创建	创建一次粗化实验	experiment 表成功生成实验记录，记录状态与 run_id 对应	是
5	图粗化运行	执行完整 UGC 流程	实验状态由 running 成功变为 completed	是
6	粗化结果展示	查看结果页	结果页成功返回并展示粗图预览、原图映射视图及相关指标区域	是
7	图可视化	点击超节点	成功返回对应 supernode 的映射子图信息，包含节点与边数据	是
8	GNN 测试	选择 GCN，训练 500 轮	成功返回 Accuracy，数据库记录训练时间	是
9	测试记录保存	完成一次 GNN 测试	gnn_test_result 表成功生成 1 条测试记录	是
10	异常输入	上传缺失标签文件的数据集	系统返回错误提示，未进入后续粗化流程	是

第四节 系统性能测试

6.4.1 页面响应性能

页面响应性能测试面向首页、配置页、运行状态页、结果展示页、图结构可视化接口和 GNN 测试页进行。考虑到图结构可视化交互实质上依赖结果页中的节点映射数据接口，因此该部分以“点击 supernode 后触发的映射接口响应时间”作为代表性测量对象。每个页面或接口连续请求 10 次，统计平均响应时间、最大响应时间及是否正常返回。测试结果如表 6.8 所示。

表 6.8 系统页面响应性能测试结果

页面/接口	平均响应时间 (ms)	最大响应时间 (ms)	是否正常返回
首页	1.372	5.989	是
配置页	3.252	4.492	是
运行状态页	13.393	75.755	是
结果展示页	149.102	228.070	是
图结构可视化接口	40.892	402.932	是
GNN 测试页	21.804	39.445	是

从测试结果可以看出，首页与配置页的响应时间均处于毫秒级，开销较小；运行状态页与 GNN 测试页在需要读取实验状态文件或数据库记录时，响应时间略有上升，但整体仍维持在较低水平；结果展示页和图结构可视化接口的耗时相对更高，其主要原因在于页面需要加载粗图预览、映射关系与图结构细节数据。

6.4.2 核心流程运行性能

核心流程运行性能测试主要考察在不同数据集规模下，系统完成一次图粗化实验与一次 GNN 测试所需的时间。为保证横向可比性，本组实验统一采用目标压缩率 50%，并记录粗化阶段总耗时与 GNN 测试耗时。测试结果如表 6.9 所示。

从表 6.9 可以看出，随着图规模与边规模的增大，粗化耗时增长较为明显。对于内置的 Cora 与 Citeseer 数据集，由于系统采用了预处理结果、预设 alpha 与 Bin Width 的快捷路径，因此整体粗化耗时较低；而 Blogcatalog 与 Flickr 需要经历数据分析、数据清洗、Alpha 计算、数据预处理、Bin Width 标定以及

表 6.9 核心流程运行性能测试结果

数据集	节点数	边数	目标压缩率	粗化耗时 (s)	GNN 测试耗时 (s)
Cora	2708	10556	50%	28.263	15.186
Citeseer	3327	9104	50%	48.501	31.795
Blogcatalog	5196	343486	50%	135.104	40.157
Flickr	7575	479476	50%	457.113	47.686

UGC 粗化的完整流程，因此耗时明显更高，尤其是 Flickr 数据集在高维稀疏特征和较大边规模下，粗化阶段开销最为突出。

另一方面，GNN 测试耗时的增长幅度相对平缓。这是因为 GNN 测试是在粗化后的图上进行训练，而粗化过程已经显著降低了图的节点规模，从而在一定程度上控制了后续模型训练开销。因此，从系统整体视角看，图粗化不仅降低了图结构规模，也为后续 GNN 实验提供了较为可控的训练时间。

第七章 总结与展望

本文围绕基于通用图粗化 (Universal Graph Coarsening, UGC) 的社交网络图粗化方法优化与系统实现展开研究, 完成了方法分析、实验验证和系统开发等工作。

本文首先梳理了 UGC 方法的基本原理与执行流程, 分析了其在社交网络数据上的适用性。针对原始 UGC 方法在部分数据集上下游分类效果不够理想的问题, 本文从超节点标签和特征聚合方式两个角度进行分析, 并采用相似度加权特征聚合方法对原始平均聚合方式进行改进。同时, 本文引入平均标签纯度和低纯度超节点占比等指标, 从标签一致性角度补充评价粗化质量。实验结果表明, 改进后的特征聚合方法能够在一定程度上提升粗化图的表示质量, 并在部分数据集上改善图神经网络分类效果。

针对社交网络中常见的多标签特点, 本文比较了多种多标签处理策略, 分析了不同策略对异配系数、粗化质量和分类结果的影响。实验结果表明, 多标签信息处理方式会影响节点相似性刻画和粗化结果质量, 说明 UGC 方法在多标签场景下仍需进一步优化。

在系统实现方面, 本文设计并实现了基于 UGC 的社交网络图粗化分析系统, 支持数据集管理、参数配置、粗化运行、结果分析、图结构可视化和 GNN 分类测试等功能, 能够支撑图粗化实验的完整流程。

总体而言, 本文验证了 UGC 方法在社交网络图粗化任务中的可行性, 并通过特征聚合优化和评价指标补充提升了实验分析的完整性。但当前工作仍存在不足, 例如多标签数据的适配仍主要停留在策略比较层面, 尚未形成贯穿节点表示、相似性度量和标签聚合的完整优化方法; 系统也主要支持单次实验运行, 对批量实验和多参数对比的支持仍不充分。

后续研究可进一步优化 UGC 方法对多标签数据集的全流程适配能力, 在节点表示构造、相似性计算和超节点标签生成过程中引入标签相关性与权重信息; 同时, 可扩展系统功能, 支持批量实验、结果自动汇总和多实验对比分析, 并增强粗化前后图结构与标签分布的可视化展示能力。

附 录

实验数据集说明

本文实验共使用七个图数据集，包括 Cora、Citeseer、Blogcatalog、Flickr、Tweibo_sampled、Facebook 和 Twitter。其中，Cora 和 Citeseer 为原 UGC 论文中使用的引文网络数据集^[12]，Blogcatalog、Flickr、Tweibo 和 Facebook 数据集来自 PyTorch Geometric 提供的 AttributedGraphDataset 数据集接口^[30]，Twitter 数据集来自 Attributed-Graph-Data 开源项目^[31]。

由于原始 tweibo 数据集规模较大，本文对该数据集进行了采样处理，并将采样后的数据集记为 Tweibo_sampled。各数据集的详细统计信息如附表 1 和附表 2 所示。

附表 1 本文使用的数据集统计信息

数据集	标签类型	图类型	节点数	边数	平均度	最大度
Cora	单标签	无向	2708	10556	7.7962	336
Citeseer	单标签	无向	3327	9104	5.4728	198
Blogcatalog	单标签	无向	5196	343486	132.2117	1538
Flickr	单标签	无向	7575	479476	126.5943	3762
Tweibo_sampled	单标签	有向	10000	559315	111.8630	2788
Facebook	多标签	无向	4039	88234	43.6910	1045
Twitter	多标签	无向	2511	37154	29.5930	190

附表 2 本文使用的数据集特征与清洗统计信息

数据集	特征维数	标签数	清洗后节点数	清洗后边数
Cora	1433	7	—	—
Citeseer	3703	6	—	—
Blogcatalog	8189	6	5196	343486
Flickr	12047	9	7575	479476
Tweibo_sampled	2000	8	10000	559315
Facebook	1283	193	2888	63373
Twitter	1323	132	1846	23961

参考文献

- [1] Newman M. Networks: an introduction[M/OL]. Oxford University Press, 2010. DOI: 10.1093/acprof:oso/9780199206650.001.0001
- [2] Hamilton W L. Graph representation learning[M]. Morgan & Claypool Publishers, 2020
- [3] Wu Z, Pan S, Chen F, *et al.* A comprehensive survey on graph neural networks[J]. IEEE Transactions on Neural Networks and Learning Systems, 2020, 32(1): 4-24
- [4] Zhou J, Cui G, Hu S, *et al.* Graph neural networks: a review of methods and applications[J]. AI Open, 2020, 1: 57-81
- [5] Hashemi M, Gong S, Ni J, *et al.* A comprehensive survey on graph reduction: sparsification, coarsening, and condensation[J]. ArXiv Preprint ArXiv:2402.03358, 2024
- [6] Chen J, Saad Y, Zhang Z. Graph coarsening: from scientific computing to machine learning [J]. SeMA Journal, 2022, 79(1): 187-223
- [7] Kernighan B W, Lin S. An efficient heuristic procedure for partitioning graphs[J]. The Bell System Technical Journal, 1970, 49(2): 291-307
- [8] Karypis G, Kumar V. A fast and high quality multilevel scheme for partitioning irregular graphs[J]. SIAM Journal on Scientific Computing, 1998, 20(1): 359-392
- [9] Loukas A. Graph reduction with spectral and cut guarantees[J/OL]. Journal of Machine Learning Research, 2019, 20(116): 1-42. <http://jmlr.org/papers/v20/18-680.html>
- [10] Ying Z, You J, Morris C, *et al.* Hierarchical graph representation learning with differentiable pooling[C]. Advances in Neural Information Processing Systems: vol. 31. Curran Associates, Inc., 2018
- [11] Cai C, Wang D, Wang Y. Graph coarsening with neural networks[J]. ArXiv Preprint ArXiv:2102.01350, 2021
- [12] Kataria M, Kumar S, *et al.* Ugc: universal graph coarsening[J]. Advances in Neural Information Processing Systems, 2024, 37: 63057-63081
- [13] 马煜昕, 徐银龙, 李成, 等. 基于输入特征稀疏化的图神经网络训练加速[J]. 计算机系统应用, 2024, 33(1): 245-253
- [14] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. ArXiv Preprint ArXiv:1609.02907, 2016
- [15] Hamilton W, Ying Z, Leskovec J. Inductive representation learning on large graphs[J]. Advances in Neural Information Processing Systems, 2017, 30
- [16] Veličković P, Cucurull G, Casanova A, *et al.* Graph attention networks[J]. ArXiv Preprint ArXiv:1710.10903, 2017

-
- [17] Xu K, Hu W, Leskovec J, *et al.* How powerful are graph neural networks?[J]. ArXiv Preprint ArXiv:1810.00826, 2018
- [18] Zhang M L, Zhou Z H. A review on multi-label learning algorithms[J]. IEEE Transactions on Knowledge and Data Engineering, 2013, 26(8): 1819-1837
- [19] Gasteiger J, Bojchevski A, Günnemann S. Predict then propagate: graph neural networks meet personalized pagerank[J]. ArXiv Preprint ArXiv:1810.05997, 2018
- [20] Cai J Y, Fürer M, Immerman N. An optimal lower bound on the number of variables for graph identification[J]. Combinatorica, 1992, 12(4): 389-410
- [21] Pallets Projects. Flask documentation[Z]. <https://flask.palletsprojects.com/en/3.0.x/>. Accessed: 2026-04-22. 2024
- [22] Paszke A, Gross S, Massa F, *et al.* Pytorch: an imperative style, high-performance deep learning library[J]. Advances in Neural Information Processing Systems, 2019, 32
- [23] Fey M, Lenssen J E. Fast graph representation learning with pytorch geometric[J]. ArXiv Preprint ArXiv:1903.02428, 2019
- [24] Franz M, Lopes C T, Huck G, *et al.* Cytoscape. js: a graph theory library for visualisation and analysis[J]. Bioinformatics, 2016, 32(2): 309-311
- [25] Li D, Mei H, Shen Y, *et al.* Echarts: a declarative framework for rapid construction of web-based visualization[J]. Visual Informatics, 2018, 2(2): 136-146
- [26] Yang Z, Cohen W, Salakhudinov R. Revisiting semi-supervised learning with graph embeddings[C]. International Conference on Machine Learning. 2016: 40-48
- [27] Shuman D I, Narang S K, Frossard P, *et al.* The emerging field of signal processing on graphs: extending high-dimensional data analysis to networks and other irregular domains[J]. IEEE Signal Processing Magazine, 2013, 30(3): 83-98
- [28] Manning C D. Introduction to information retrieval[M]. Syngress Publishing, 2008
- [29] Jaccard P. Etude de la distribution florale dans une portion des alpes et du jura[J/OL]. Bulletin De La Societe Vaudoise Des Sciences Naturelles, 1901, 37: 547-579. DOI: 10.5169/seals-266450
- [30] PyTorch Geometric Team. AttributedGraphDataset[Z]. https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.AttributedGraphDataset.html. Accessed: 2026-04-24. 2024
- [31] He T. Attributed Graph Data[Z]. <https://github.com/he-tiantian/Attributed-Graph-Data>. Accessed: 2026-04-24. 2020

致 谢

在南开的四年，海棠花开了四次，又落了四次。来去匆匆间，我似乎很少真正驻足欣赏，于是花开花落带走了四年的时光，我的本科生涯也即将画上句点。回望这段求学之路，心中满是感念与不舍。在此，谨向所有给予我关怀、帮助与支持的师长、亲友致以最诚挚的谢意。

衷心感谢温延龙老师、宋春瑶老师以及戚晓睿学长。从论文的选题立意、实验方案设计，到文稿的打磨与完善，每一个阶段、每一处细节都离不开老师们与学长的悉心指导与耐心帮助。完成本篇毕业论文的过程，既是对我四年本科所学知识的系统梳理与沉淀，更是一次珍贵的科研启蒙经历。诸位师长渊博的学识与严谨的科研态度，使我受益匪浅。

感谢南开大学计算机学院和密码与网络空间安全学院的全体授课教师。正是各位老师在课堂上的倾囊相授，使我逐步建立起较为扎实的专业基础，也为我后续的学业深造与人生发展奠定了坚实基础。

感谢朝夕相伴的同窗与舍友，感恩能与大家共度这段珍贵的大学时光。特别感谢何禹姗、刘俊彤、刘玉菡等好友，我们一同学习、成长，共度岁岁朝夕，这段真挚美好的同窗情谊，值得我一生珍藏。

亦感谢挚友李美英，每当我陷入迷茫犹豫、遭遇困顿挫折之时，始终有你的陪伴、鼓励与相助，给予我温暖与力量。这份珍贵的情谊，我将永远铭记于心。

深深感恩我的家人。无论我身处顺境还是逆境，你们始终理解我的选择、尊重我的决定，以默默的关怀与长久的陪伴，做我求学路上最坚实的后盾。是你们的包容、牵挂与无私付出，给予了我一往无前的勇气。

感谢我的恋人王金源，这个几乎陪伴了我整个学生时代的人。你陪我走过最多的地方，见过我最多的眼泪，也替我分担了最多的压力与焦虑。感谢你的爱、支持与包容。无论前路风雨几何，能与你并肩同行，见证彼此的成长，我深感幸运。

最后，感谢一路走来始终坚持、奋力前行的自己。感谢那些披星戴月的时光，也感谢那些曾让我迷茫与流泪的过往。它们共同塑造了现在的我——这个仍有许多不足，却也令自己感到满意的我。前路漫漫，愿往后的自己依旧心怀热忱，步履不停，奔赴下一场山海。